Operating Systems Exercises

姓名：杨云帆
学号：2400012916

Lesson 01 - Execution model

Question 1 - State trace

QUESTION

On a single-core system, initially A is **running in user mode** and B is **ready**. The following events occur:

1. A calls read(); the requested data is not in memory and requires disk I/O.
2. The kernel starts the I/O.
3. The scheduler selects B.
4. While B runs in user mode, the disk completes and interrupts the CPU.
5. The kernel decides to continue running B after handling the interrupt.
6. Later, B's time quantum expires and the scheduler selects A.

For each event, identify CPU mode, the states of A and B, and whether a context switch occurs. Treat these as events/transitions, not six stable stages: distinguish the state before, during, and after an event where necessary.

ANSWER

(1) A invokes `read()` in user mode, causing a trap into kernel mode. A remains Running while the kernel handles the system call; it has not been blocked or switched out yet. B remains Ready.

(2) The kernel discovers that the requested data is not in the page cache, submits a disk I/O request, and blocks A. At the end of this event, A is Blocked, B is Ready, and the CPU is executing the scheduler in kernel mode.

(3) The scheduler selects B and performs a context switch from A to B. While the switch is being performed, the CPU remains in kernel mode. After the return from trap, B runs in user mode; A is Blocked.

(4) While B is running in user mode, the device raises an I/O-completion interrupt. The CPU enters kernel mode in B's execution context, and the interrupt handler moves A from Blocked to Ready. This does not necessarily cause a context switch.

(5) The kernel decides to continue running B and returns to user mode. A is Ready and B is Running. No thread context switch occurs.

(6) B's time quantum expires, causing a timer interrupt rather than a trap. The CPU enters kernel mode, the scheduler moves B from Running to Ready and selects A, and a context switch occurs. After returning to user mode, A is Running and B is Ready.

One issue with the original question is that the six items describe events or transitions rather than stable stages. The processor mode may change within a single item, so each item should specify the state before the event, during kernel handling, and after the event.

Question 2 - Mode switch, blocking, and context switch

QUESTION

For each event below, say whether it enters the kernel, whether it can block, and whether it **necessarily** causes a thread context switch. Distinguish “possible” from “inevitable.”

1. A getpid() system call.
2. A timer interrupt.
3. A blocking read() that misses in the disk/page cache.

4. Acquiring an uncontended user-space mutex.
5. Acquiring a mutex held by another thread.
6. Accessing a valid virtual address whose page is not currently resident in physical memory.

ANSWER

(1) `getpid()` enters the kernel through a system call. It does not normally block and does not require a context switch.

(2) A timer interrupt enters the kernel if the CPU was executing in user mode. It does not block the interrupted thread, although the scheduler may preempt that thread and perform a context switch.

(3) A blocking `read()` that encounters a disk page-cache miss enters the kernel through the `read` system call, not through a page fault. The calling thread normally blocks while waiting for I/O, so the scheduler must run another runnable thread or the idle thread.

(4) Acquiring an uncontended user-space mutex normally succeeds through an atomic fast path without entering the kernel. It neither blocks nor causes a context switch.

(5) Acquiring a contended mutex depends on its implementation. A spinlock may remain entirely in user mode and busy-wait, whereas a futex-based mutex may enter the kernel and block. A context switch is required only if the current thread blocks or is independently preempted.

(6) Accessing a valid virtual address whose page is not currently mapped causes a page-fault exception and enters the kernel. It may be resolved immediately, such as through demand-zero allocation or copy-on-write, or it may require I/O and block. Therefore, neither blocking nor a context switch is inevitable.

Question 3 - Shared versus thread-private state

QUESTION

Two threads, T1 and T2, belong to the same process. Classify each item as shared or thread-private, and briefly explain why:

1. Global variables
2. Heap objects
3. Ordinary local variables
4. The file-descriptor table
5. The program counter (PC)
6. Page tables
7. User stacks
8. Kernel stacks

ANSWER

(1) Shared. Ordinary global variables reside in the process's shared address space, commonly in `.data`, `.bss`, or `.rodata`. A `thread_local` variable is an exception and has one instance per thread.

(2) Shared. The heap belongs to the process's shared address space, so every thread can access a heap object if it has a pointer to it.

(3) Logically private. Ordinary automatic local variables are stored in a thread's register context or user stack. However, the stack is still mapped in the shared address space, so another thread can access such a variable if its address is published and its lifetime has not ended.

(4) Shared. POSIX threads normally share the process's file descriptor table. For example, closing a descriptor in one thread affects the other threads.

(5) Private. Each thread has its own logical program counter and register context. When the thread is not running, these values are saved in its thread control block rather than occupying dedicated physical CPU registers.

(6) Shared. Threads in the same process share the same virtual address space and therefore the same page-table hierarchy.

(7) Private. Each thread has its own user stack and stack pointer, although its stack memory remains addressable by other threads in the same process.

(8) Private. Each kernel-schedulable thread normally has its own kernel stack so that its system-call, exception, or interrupt-handling call chain can be suspended and later resumed independently.

Question 4 - User-pointer TOCTOU

QUESTION

For `write(fd, user_buffer, n)`, the kernel checks that `user_buffer` is valid before copying. Between that check and the copy, another thread in the same process unmaps the buffer.

1. What problem does this expose?
2. Why is a single initial pointer check insufficient?
3. Suggest a safe handling strategy.

ANSWER

(1) This is a time-of-check-to-time-of-use (TOCTOU) race across the user-kernel trust boundary. `user_buffer` may be located on the heap, stack, in static storage, or in an `mmap` region. Another thread can change or unmap that memory after the kernel validates it but before the kernel uses it.

(2) A one-time validation is insufficient because the address-space mapping can change between validation and copying. This may happen after a context switch on a single-core machine or concurrently on another CPU core, without any context switch on the current core.

(3) The kernel cannot rely on a user-space mutex because an erroneous or malicious thread may ignore it. Instead, the kernel should use a fault-safe mechanism such as `copy_from_user()` and copy the data into a kernel buffer before performing later I/O. For asynchronous I/O or DMA, it may need to pin the relevant pages or otherwise hold the necessary virtual-memory locks or references while the device uses them.

Lesson 02 - Unix processes

Question 1 - Process count

QUESTION

Assume every `fork()` succeeds:

```
fork();

if (fork() == 0) {
    fork();
}

printf("X");
```

How many processes exist at the end? How many `X` characters are printed? Draw the process-creation tree.

ANSWER

There are six processes, so `X` is printed six times. After the first `fork()`, two processes execute the second `fork()`. The two children created by the second `fork()` satisfy the condition and each executes the third `fork()`.

```
P0
├── P1                created by the first fork
│   └── P3            created by P1's second fork
│       └── P5        created by P3's third fork
└── P2                created by P0's second fork
    └── P4            created by P2's third fork
```

Question 2 - Output ordering

QUESTION

Assume each one-byte `write()` is atomic:

```
write(1, "A", 1);

pid_t pid = fork();

if (pid == 0) {
    write(1, "B", 1);
    _exit(0);
}

write(1, "C", 1);
waitpid(pid, NULL, 0);
write(1, "D", 1);
```

List every possible complete output and explain the ordering constraints.

ANSWER

possible outputs:

```
ABCD
ACBD
```

`write(1, "D", 1)` executes only after `waitpid()` has reaped the child, so `D` appears after `B`. It also appears after `C` because of the parent's program order. The only unordered pair is `B` and `C`.

Question 3 - Successful `exec()`

QUESTION

Assume `exec()` succeeds:

```
printf("A\n");

pid_t pid = fork();

if (pid == 0) {
    execl("/bin/echo", "echo", "B", NULL);
    printf("C\n");
} else {
    waitpid(pid, NULL, 0);
    printf("D\n");
}
```

Which letters appear, which do not, what ordering is guaranteed, and does the child's PID change across `exec()`? Distinguish the logical ordering from any effect of stdio buffering.

ANSWER

`A`, `B`, and `D` appear, whereas `C` does not. A successful `exec()` replaces the child's program image and never returns to the following `printf`. The child's PID does not change.

If standard output is a terminal and `A\n` is flushed before `fork()`, the output is:

```
A
B
D
```

If standard output is fully buffered, such as when redirected to a file, the visible order can instead be `B`, `A`, `D`: the child's inherited stdio buffer is discarded by the successful `exec()`, while the parent eventually flushes its own buffer. The invariant that matters here is that `B` appears before `D`.

Question 4 - Shared file offset

QUESTION

The file `data.txt` contains `ABCDEF`. The program opens it before `fork()`:

```
int fd = open("data.txt", O_RDONLY);
pid_t pid = fork();

char buf[3] = {0};

read(fd, buf, 2);
printf("%s\n", buf);
```

Both parent and child execute one `read()`. Ignore `printf` interleaving and assume each regular-file read and shared-offset update is atomic. Can they both read `AB`? Which pair of substrings can they obtain? Why is ordinary memory private while the file offset is shared?

ANSWER

The parent and child cannot both read `AB`. They have separate file descriptor table entries, but those entries refer to the same kernel open file description and therefore share its current file offset.

Assuming the regular-file `read` and file-offset update are atomic, the two reads return `AB` and `CD`. Which process receives which substring is nondeterministic:

```
parent: AB, child: CD
```

or

```
child: AB, parent: CD
```

Combinations such as `AC` and `BD` are impossible because each single `read(fd, buf, 2)` consumes two consecutive bytes and advances the shared offset by two.

Lesson 03 - Atomicity and locks

Question 1 - Lost updates

QUESTION

Initially:

```
int x = 0;
```

Thread A performs two increments:

```
x++;
x++;
```

Thread B performs one increment:

```
x++;
```

Assume every increment consists of three individually atomic steps:

```
load x
add 1 locally
store x
```

The operations within each thread preserve program order, but the scheduler may interleave the two threads arbitrarily.

1. What final values of x are possible?
2. Give one valid execution trace for every possible final value.
3. Can the final value be 0? Explain.
4. Can the final value be greater than 3? Explain.

ANSWER

(assume x is global) 1, 2, 3 are all possible;

```
l: load, a: add, s: store

case 1:
thread A: -laslas--
thread B: l------as
glob val: 000111221
result: x=1
```

```
case 2:
thread A: la--s-las
thread B: --la-s---
glob val: 000011112
result: x=2

case 3:
thread A: laslas---
thread B: ------las
glob val: 001112223
result: x=3
```

0 is not possible because the final 'store x' operation will increase the value of x by one

value greater than 3 is also not possible because there are at most three 'add 1 locally' atomic instructions

Question 2 - Check-then-act

QUESTION

Two threads execute:

```
bool reserve_seat(void) {
    if (available_seats == 0)
        return false;

    available_seats--;
    return true;
}
```

Initially:

```
available_seats = 1
```

1. Give an interleaving in which both calls return true.
2. What is the final value of available_seats?
3. Identify the invariant that has been violated.
4. Rewrite the function using a mutex.
5. Explain why locking only around available_seats-- is insufficient.

ANSWER

```
thread 1: check ->                -> decre -> ret
thread 2:          check -> decre                -> ret
```

available_seats = 0 or -1

invariant: available_seats >= 0

```
bool reserve_seat(void) {
    acquire(&myLock);
    if (available_seats == 0) {
        release(&myLock);
        return false;
    }

    available_seats--;
    release(&myLock);
    return true;
}
```

Locking only around `available_seats--` is insufficient because it does not change the fact that check and action are not done atomically together; action takes place after the check. the check-then-act pattern stays unchanged.

Question 3 - Spin or block?

QUESTION

For each situation, choose a spinlock, a blocking mutex, or neither. Explain your reasoning.

1. A kernel critical section modifies two pointers and takes about 30 nanoseconds.

2. A user thread protects a data structure while performing a network request that may take 200 milliseconds.
3. A thread waits for another thread to produce an item in an empty queue.
4. A multicore kernel protects a scheduler run queue for a very short operation.
5. A single-threaded program updates a private local variable.

Do not answer only with performance arguments; also consider whether the thread is allowed to sleep and whether it is waiting for a condition rather than exclusive ownership.

ANSWER

(1) Spinlock. The kernel critical section is extremely short, so the expected waiting time is smaller than the overhead of putting the thread to sleep and waking it up. Code must not sleep while holding a spinlock.

(2) Blocking mutex. A 200 ms network request is far too long for busy waiting. Better still, if the design permits it, copy or reserve the required state, release the lock, and then perform the network request instead of holding any lock across blocking I/O.

(3) Block. The consumer is waiting for a condition rather than merely waiting for a short critical section. A mutex alone is not sufficient: normally, the queue is protected by a mutex and the consumer waits using a condition variable (or a semaphore) until an item may be available.

(4) Spinlock. On a multicore kernel, a spinlock is suitable for a very short operation on a shared run queue, provided that the holder cannot sleep.

(5) Neither. Thread-private local data is not shared, so no synchronization is required.

The roles of these mechanisms are different: a mutex protects a critical section, a condition variable allows a thread to sleep until a condition may have changed, and a spinlock is appropriate only when the expected wait is very short and the lock holder can continue running.

Question 4 - Broken lock

QUESTION

Consider:

```
void acquire(Lock *L) {
    while (L->held) {
    }

    L->held = true;
}

void release(Lock *L) {
    L->held = false;
}
```

1. Show an interleaving in which two threads both enter the critical section.
2. Why does declaring held as volatile not fix the problem?
3. What property must the hardware primitive provide?
4. Rewrite acquire using an atomic exchange operation.

ANSWER

(1)

```
// Initially, L->held == false.

thread A                          thread B
read L->held -> false
                                  read L->held -> false
write L->held = true
                                  write L->held = true
enter the critical section        enter the critical section

// Both threads enter because both reads occur before either write.
```

(2) Declaring `held` as `volatile` does not fix the lock. `volatile` may force the program to perform the memory accesses, but it does not combine the check and the update into one indivisible operation. Thread B can still read `false` before

thread A stores `true`, so the check and the action remain split. `volatile` therefore does not provide mutual exclusion or proper thread synchronization.

(3) The hardware primitive must perform an **atomic read-modify-write** operation: no other processor may observe or modify the memory location between its read and its write. `test&set`, `swap`, and `compare&swap` are examples of such primitives.

(4)

```
void acquire(Lock *L) {
    while (test&set(&L->held, true)) {
        // spin while the old value is true
    }
}

void release(Lock *L) {
    L->held = false;  // CS162 abstraction
}
```

Here, `test&set` atomically stores `true` in `L->held` and returns its old value. Exactly one competing thread can observe an old value of `false` and leave the loop.

Question 5 - Interrupts and multiple cores

QUESTION

A kernel developer proposes:

```
disable_interrupts();
shared_counter++;
enable_interrupts();
```

1. Why might this protect shared_counter on a single-core machine?
2. Why is it insufficient on a multicore machine?
3. Why must ordinary user programs not be allowed to disable interrupts?
4. What could go wrong if an interrupt handler also needs a lock currently held by the interrupted kernel code?

ANSWER

(1) Disabling interrupts prevents interrupt-driven preemption on the current CPU, assuming the kernel code does not voluntarily block or yield. The protection applies to the entire critical section; it is not based on `shared_counter++` being one machine instruction, because an increment is normally a load/add/store sequence.

(2) Disabling interrupts affects only the current CPU. Threads executing concurrently on other CPUs can still read or modify `shared_counter`, so disabling interrupts alone does not provide mutual exclusion on a multicore machine.

(3) A faulty or malicious user program that could disable interrupts could prevent timer interrupts from returning control to the kernel, monopolize the CPU, and delay device handling. Therefore, disabling interrupts is a privileged kernel operation.

(4) A self-deadlock can occur. If the interrupt handler waits for a lock held by the interrupted kernel code on the same CPU, the holder cannot resume and release the lock until the interrupt handler returns, while the handler cannot return until it acquires the lock.

Question 6 - Locking invariant

QUESTION

A doubly linked list maintains:

```
node.next.prev == node
node.prev.next == node
```

A removal operation updates several pointers.

1. Why must the entire removal operation be protected rather than each pointer assignment being locked separately?
2. What inconsistent state could another thread observe?

3. Should the lock conceptually protect individual pointer variables or the list invariant?
4. What determines whether one global list lock or one lock per node is appropriate?

ANSWER

(1) Atomic pointer assignments do not make the complete list operation atomic. Removing a node requires multiple related pointer updates, and another thread can observe or modify the list between those updates. For example, while adjacent nodes `node2` and `node3` are being removed concurrently from `node1 <-> node2 <-> node3 <-> node4`, one operation can update `node1.next` while the other uses stale adjacency information. The result may contain inconsistent links such as `node1.next.prev != node1`. The entire logical update, rather than each individual assignment, must therefore be synchronized.

(2) For every node in a valid doubly linked list, the local structural invariant is:

```
if (node->next != NULL)
    node->next->prev == node;

if (node->prev != NULL)
    node->prev->next == node;
```

With sentinel nodes, the implementation can often avoid the `NULL` boundary cases.

(3) The critical section must protect the complete transition from one valid list state to another valid list state. No other thread should observe or interfere with an intermediate state in which the list invariant is temporarily false.

(4) The appropriate lock granularity depends on the operation and the required concurrency. A single global list lock is simple and correct, but it limits concurrency. Structural modifications do not necessarily require locking the entire list: a fine-grained implementation can lock only the affected nodes, provided it acquires locks in a consistent order, revalidates the relevant adjacency relationships when necessary, and protects node lifetime so that a node cannot be freed while another thread is using it. A per-node lock can protect a node's payload when the node cannot concurrently be removed and freed.

Lesson 04 - Blocking synchronization

Shared assumptions: Assume Mesa condition-variable semantics and the nonnegative semaphore model used in the lesson.

Question 1 - Fundamental properties

QUESTION

For each statement, determine whether it is true or false and explain why.

1. If signal(&cv) executes when nobody is waiting, the next thread that calls wait(&cv, &lock) will immediately return.
2. A thread holds lock when wait(&cv, &lock) returns.
3. A binary semaphore is always interchangeable with a mutex.
4. After broadcast(&cv), all awakened threads enter the critical section simultaneously.
5. while is needed around wait only because operating systems may generate spurious wakeups.
6. It is legal and useful for a thread to sleep in wait while retaining the associated lock.

ANSWER

(1) False. A condition variable is different from a semaphore: it does not remember signals. If no thread is waiting, `signal(&cv)` is forgotten, and a thread that later calls `wait(&cv, &lock)` will still block.

(2) True. A thread releases the lock while waiting and reacquires it before `wait()` returns.

(3) False. A binary semaphore can be used to provide mutual exclusion, but it is not always interchangeable with a mutex. A mutex normally has an owner and must be released by that owner. A semaphore represents permits and can also be used for signaling, with one thread performing `P` and another performing `V`.

(4) False. The awakened threads must compete to reacquire the lock, and only one thread can execute in the critical section at a time.

(5) False. The `while` loop is required by Mesa semantics, not merely because of possible spurious wakeups. `signal()` wakes at most one waiter, whereas `broadcast()` may wake multiple waiters. In either case, an awakened thread does not immediately run: before it reacquires the lock, another thread may change the shared state and make the predicate false again. A signal therefore means only that a waiting thread should try again, not that it is guaranteed to proceed.

(6) False. `wait()` must release the associated lock while the thread sleeps. Otherwise, a thread that needs the lock to change the predicate cannot proceed, which can cause deadlock.

Question 2 - Why `while`, not `if`?

QUESTION

Consider:

```
Item get(void) {
    lock_acquire(&lock);

    if (count == 0) {
        wait(&not_empty, &lock);
    }

    Item item = remove();
    count--;

    lock_release(&lock);
    return item;
}
```

Initially, `count == 0`.

Threads `C1` and `C2` are consumers, and `P` is a producer.

1. Construct an execution in which C1 waits, P produces exactly one item and signals C1, but C2 removes the item before C1 reacquires the lock.
2. What happens when C1 returns from wait?
3. Which safety property is violated?
4. Correct the implementation.

ANSWER

(1)

```
C1: acquire lock
C1: observe count == 0
C1: wait -> atomically release lock and sleep

P:  acquire lock
P:  insert one item; count = 1
P:  signal C1
P:  release lock
    C1 is ready but has not reacquired the lock

C2: acquire lock
C2: remove the item; count = 0
C2: release lock

C1: reacquire lock and return from wait
C1: continue past the if
C1: remove from the empty buffer; count = -1
```

(2) `C1` continues past the `if` without checking the predicate again. It executes `remove()` on an empty buffer and decrements `count` to `-1`.

(3) The safety property "a consumer must never remove from an empty buffer" is violated. Equivalently, the invariant `0 <= count <= N` is violated.

(4) The correct implementation is:

```
Item get(void) {
    lock_acquire(&lock);

    while (count == 0) {
```

```
        wait(&not_empty, &lock);
    }

    Item item = remove();
    count--;

    lock_release(&lock);
    return item;
}
```

Question 3 - Lost wakeup

QUESTION

A student implements waiting as follows:

```
lock_acquire(&lock);

while (!ready) {
    lock_release(&lock);
    sleep_on(&cv);
    lock_acquire(&lock);
}

use_resource();
lock_release(&lock);
```

Here, `sleep_on` only puts the thread on `cv`'s waiting queue; it does not release any lock.

1. Construct an execution involving a waiter and a signaler that causes the waiter to sleep forever even though ready == true.
2. Identify the vulnerable interval.
3. Explain precisely how wait(&cv, &lock) eliminates this race.
4. Why is it incorrect to fix the program by moving lock_release after sleep_on?

ANSWER

(1)

```
Consumer: acquire lock
Consumer: observe ready == false
Consumer: release lock

Producer: acquire lock
Producer: set ready = true
Producer: signal(cv)       // no waiter is registered, so the signal is lost
Producer: release lock

Consumer: sleep_on(cv)     // may now sleep forever
```

(2) vulnerable interval:

```
lock_release(&lock);
// vulnerable: a signal can occur before the thread joins cv's wait queue
sleep_on(&cv);
```

(3) `wait(&cv, &lock)` atomically releases the lock and registers the thread on the condition variable's waiting queue. There is therefore no interval in which the waiter has released the lock but is not yet visible to a signaler.

(4) Moving `lock_release()` after `sleep_on()` causes deadlock. Because `sleep_on()` blocks, the consumer cannot reach `lock_release()`. The producer cannot acquire the lock to set `ready` and signal the consumer.

Question 4 - Semaphore ordering and deadlock

QUESTION

A producer incorrectly uses:

```
void put(Item item) {
    P(&mutex);
    P(&empty);
```

```
    insert(item);

    V(&full);
    V(&mutex);
}
```

A consumer uses:

```
Item get(void) {
    P(&mutex);
    P(&full);

    Item item = remove();

    V(&empty);
    V(&mutex);
    return item;
}
```

1. Starting with a full buffer, construct a deadlock execution.
2. Starting with an empty buffer, construct the symmetric deadlock execution.
3. Rewrite both operations correctly.
4. State a general rule about acquiring a mutual-exclusion lock before performing an operation that may block.

ANSWER

(1)

```
     operation      mutex     full     empty
00: Prod: P(&mutex)  1->0       n        0
01: Prod: P(&empty)   0         n        0
    // producer get blocked because there's no empty slots
02: Cons: P(&mutex)   0         n        0
    // consumer get blocked because mutex is held by producer
    // nobody can proceed, deadlock takes place
```

(2)

```
     operation      mutex     full     empty
00: Cons: P(&mutex)  1->0       0        n
01: Cons: P(&full)    0         0        n
    // consumer get blocked because there's no filled slots
02: Prod: P(&mutex)   0         0        n
    // producer gets blocked because mutex is held by the consumer
    // nobody can proceed, deadlock takes place
```

(3)

```
void put(Item item) {
    P(&empty);
    P(&mutex);

    insert(item);

    V(&mutex);
    V(&full);
}

Item get(void) {
    P(&full);
    P(&mutex);

    Item item = remove();

    V(&mutex);
    V(&empty);
    return item;
}
```

Reversing each pair of `V` operations would also preserve correctness because `V` does not block. Releasing `mutex` first is conventional because it avoids waking a thread only for that thread to immediately block on `mutex`.

(4) Acquire the mutual-exclusion lock before entering the critical section, and hold it only for the protected operation. Do not hold a mutual-exclusion lock while executing an operation that may block indefinitely, unless that operation atomically releases the lock as part of the synchronization protocol, as `wait(cv, lock)` does. In this bounded-buffer protocol, wait

on the resource semaphore (`empty` or `full`) before acquiring `mutex`.

Question 5 - One-shot barrier

QUESTION

Implement:

```
void barrier(void);
```

There are exactly `N` participating threads. No thread may return from `barrier` until all `N` threads have arrived. The barrier is used only once.

You may use:

```
Lock lock;
Condition all_arrived;
int arrived = 0;
```

Answer the following:

1. Write barrier.
2. State an invariant involving arrived.
3. Explain why broadcast is needed instead of signal.
4. Explain why waiting threads must use while.
5. Would your implementation still work if the barrier were reused? If not, briefly identify the new problem; you do not need to implement a reusable barrier yet.

ANSWER

(1)

```
void barrier(void) {
    acquire(&lock);
    arrived++;
    if (arrived == N) {
        broadcast(&all_arrived);
    }
    while (arrived < N) {
        wait(&all_arrived, &lock);
    }
    release(&lock);
}
```

(2) `0 <= arrived <= N`. More strongly, `arrived` equals the number of participating threads that have entered this barrier, and no thread returns while `arrived < N`.

(3) signal wakes up a single waiting thread, while broadcast signals all waiting threads to wake up. In the current question, we want to wake up all waiting threads.

(4) In general, Mesa-style condition-variable waits use `while` because an awakened thread does not run immediately and must recheck its predicate after reacquiring the lock. There is a subtle exception here: in this one-shot barrier, `arrived` never decreases, so once `arrived == N`, the predicate cannot become false again. Under an abstract model with no spurious wakeups, `if` would therefore be sufficient for this particular barrier. `while` remains the conventional and robust form because it handles spurious wakeups and follows the standard predicate-waiting discipline.

(5) No. If `arrived` remains equal to `N`, every later call passes immediately. Simply resetting it to zero is also unsafe: threads awakened in the previous round may reacquire the lock after the reset, observe `arrived < N`, and incorrectly wait as if they belonged to the next round. A reusable barrier needs a generation or phase number to distinguish successive rounds.

Lesson 05 - Synchronization policy and fairness

Shared assumptions: Assume Mesa condition-variable semantics.

Question 1 - Classifying properties

QUESTION

Classify each property as primarily **safety**, **liveness**, or **fairness**. Explain briefly.

1. No two writers execute concurrently.
2. A writer never executes concurrently with a reader.
3. Every thread waiting for a lock eventually acquires it.
4. The program cannot reach a state in which every participating thread is permanently blocked.
5. After thread A requests a lock, at most ten later requests may acquire it before A.
6. 0 <= active_readers.

Then explain the difference between deadlock and starvation using one short execution scenario for each.

ANSWER

(1) Safety. The invariant is that at most one writer may be active: `active_writers <= 1`.

(2) Safety. A reader must not execute concurrently with a writer. Otherwise, it may observe a partially completed update while the protected data structure's invariant is temporarily false.

(3) Fairness, specifically starvation freedom. It is also a liveness property in the broader theoretical sense because it requires that every waiting thread eventually make progress.

(4) Liveness, specifically deadlock freedom. The participating threads must not reach a state in which none of them can make progress.

(5) Fairness, specifically bounded waiting. After A requests the lock, at most ten later requests may acquire it before A.

(6) Safety. The counter never becomes negative; this is a state invariant.

deadlock: thread 1 holds a lock A and requests for lock B, but get blocked because lock B is owned by thread 2; similarily, thread 2 holds lock B and requests for lock A, but get blocked because lock A is owned by thread 1. nobody can proceed.

starvation: In a reader-preference readers-writer system with a continuous flow of overlapping readers, there is always at least one active reader. Individual readers finish and the system keeps making progress, but a waiting writer never gets a chance to write. The writer starves.

Question 2 - Writer starvation

QUESTION

Consider a reader-preference implementation in which a reader waits only when `writer_active == true`.

Initially, there are no active readers or writers.

1. Construct an execution involving writer W and readers R1, R2, … in which W waits indefinitely.
2. Ensure that every individual reader eventually completes.
3. Explain why this execution is starvation rather than deadlock.
4. Identify the predicate that readers must additionally check to implement writer preference.

ANSWER

(1) use (Ai, Di) to describe a reader's task: Ai: arrival time, Di: duration time R1: (0, 2), R2: (1, 3), ... Ri: (i - 1, i + 1), ...; W: (1, 3) In this case, consecutive readers overlap, so `W` waits indefinitely even though every individual reader completes.

(2) (already realized)

(3) This is starvation rather than deadlock because reader threads remain active and continue to complete. Although `W` waits indefinitely, the system as a whole continues making progress.

(4) Readers must additionally check `waiting_writers > 0`. The complete writer-preference predicate is:

```
while (writer_active || waiting_writers > 0) {
    wait(&can_read, &lock);
}
```

Question 3 - Debugging a readers-writer monitor

QUESTION

Consider:

```
void begin_write(void) {
    acquire(&lock);

    while (writer_active || active_readers > 0) {
        waiting_writers++;
        wait(&can_write, &lock);
        waiting_writers--;
    }

    writer_active = true;
    release(&lock);
}
```

1. Why is updating waiting_writers inside the loop problematic?
2. What happens if the writer wakes several times before it can proceed?
3. Rewrite begin_write correctly.
4. Write the corresponding reader admission predicate for writer preference.
5. Can strict writer preference starve readers? Give an execution pattern.

Review clarification: Parts 1-2 were asked with a false premise. Under the stated Mesa-monitor abstraction, incrementing before each wait and decrementing after each return need not miscount: the writer holds the lock during the transient interval. The revised answer should distinguish awkward bookkeeping from an actual correctness bug.

ANSWER

(1) Under the CS162 monitor model, the original implementation is awkward but not incorrect. Every execution of `waiting_writers++` before `wait()` is matched by `waiting_writers--` when that call to `wait()` returns. If the predicate is still false, the next loop iteration increments the counter again before releasing the lock and sleeping.

While the awakened writer is temporarily not counted, it holds `lock`, so no other monitor operation can observe that temporary value. Thus, `waiting_writers` correctly counts the blocked writers whenever another thread can inspect it.

(2) If the writer wakes several times before it can proceed, each wakeup decrements the counter once and each subsequent wait increments it once. The counter therefore does not accumulate an extra count. The conventional implementation below is nevertheless clearer because each distinct writer increments once before the loop and decrements once after it is admitted.

(3)

```
void begin_write(void) {
    acquire(&lock);
    waiting_writers++;

    while (writer_active || active_readers > 0) {
        wait(&can_write, &lock);
    }

    waiting_writers--;
    writer_active = true;
    release(&lock);
}
```

(4)

```
void begin_read(void) {
    acquire(&lock);

    while (writer_active || waiting_writers > 0) {
        wait(&can_read, &lock);
    }

    active_readers++;
    release(&lock);
}
```

(5) yes. Every time the lock becomes idle, either `waiting_writers > 0` already (the next writer is queued), so R1's predicate still holds and it re-waits - or a new writer arrives before R1 gets scheduled to re-check.

Question 4 - Broken reusable barrier

QUESTION

A student writes:

```
void barrier(void) {
    acquire(&lock);

    arrived++;

    if (arrived == N) {
        arrived = 0;
        broadcast(&all_arrived);
    } else {
        while (arrived != 0) {
            wait(&all_arrived, &lock);
        }
    }

    release(&lock);
}
```

For `N == 2`:

1. Construct an execution where T1 is still completing the first barrier round while T2 quickly enters the second round.
2. Show how T1 can mistake the second round's arrived value for an incomplete first round and sleep again.
3. Explain why this may deadlock.
4. Correct the barrier using a generation counter.
5. State the predicate on which a thread should wait.

ANSWER

Parts (1)-(2).

```
T1: enter the first barrier; arrived = 1; wait and release lock
T2: enter the first barrier; arrived: 1 -> 2 -> 0; broadcast
T2: release lock and return from the first barrier
T2: enter the second barrier; arrived: 0 -> 1; wait and release lock
T1: reacquire lock after the old broadcast
T1: observe arrived == 1 and wait again, although it is still completing round one
```

(3) `T1` is still trapped in its first call, so it cannot enter the second round and increment `arrived`. Both threads wait for a broadcast that can occur only after two second-round arrivals, resulting in deadlock.

(4)

```
void barrier(void) {
    acquire(&lock);

    int current_generation = generation_counter;
    arrived++;

    if (arrived == N) {
        arrived = 0;
        generation_counter++;
        broadcast(&cv);
    } else {
        while (generation_counter == current_generation) {
            wait(&cv, &lock);
        }
    }

    release(&lock);
}
```

(5) The waiting predicate is `generation_counter == current_generation`. A thread waits while its recorded generation is still the current generation and proceeds once the global generation counter changes.

Question 5 - Signaling a resource pool

QUESTION

Initially:

```
available = 0
three threads are waiting
```

A releasing thread executes:

```
acquire(&lock);
available += 2;
signal(&resource_available);
release(&lock);
```

1. How many waiting threads are guaranteed to be awakened?
2. After the awakened thread consumes one resource, what values and thread states may remain?
3. Is memory safety necessarily violated?
4. Which liveness problem can occur?
5. Give two correct signaling strategies.
6. Compare their performance implications.

ANSWER

(1) Exactly one of the three waiting threads is awakened.

(2) Under the stated closed scenario, the awakened thread consumes one resource and completes, `available == 1`, and two threads remain asleep. If additional consumers are allowed to barge before the awakened thread reacquires the lock, other states are possible, but that requires an execution beyond the one specified in the question.

(3) Memory safety is not necessarily violated. Assuming each consumer holds the lock and checks `available` in a `while` loop, no consumer uses a nonexistent resource.

(4) A liveness and resource-utilization problem can occur: one permit remains available while eligible threads remain asleep because they were not signaled. This is insufficient signaling, leaving a stranded waiter; it is not itself a lost-wakeup race.

(5) Two correct strategies are to call `signal()` once for each new resource-in this case, twice-or to call `broadcast()` once.

(6) Two calls to `signal()` wake only the two threads that can consume resources and therefore avoid unnecessary work. `broadcast()` is simpler, but it wakes all three waiters; after two consume the available resources, the third must reacquire the lock, find the predicate false, and sleep again. This unnecessary competition is a small thundering-herd effect.

Lesson 06 - CPU scheduling fundamentals

Shared assumptions: Unless stated otherwise: one CPU; zero context-switch cost; no job I/O; alphabetical tie-breaking; a new arrival is schedulable at its arrival time.

Question 1 - FCFS and SJF

QUESTION

Consider:

Job	Arrival	CPU burst
A	0	6
B	1	3
C	2	1
D	4	2

For both FCFS and non-preemptive SJF:

1. Draw the Gantt chart.
2. Calculate each job's response time.
3. Calculate each job's turnaround time.
4. Calculate each job's waiting time.
5. Calculate the average of each metric.
6. Explain why SJF cannot preempt A in this example.

ANSWER

(1)

```
FCFS:
Time: 0          6      9  10     12
Job : |    A     |  B   | C |  D   |

SJF:
Time: 0          6    7      9     12
Job : |    A     | C |  D   |  B   |
```

(2)

response time	A	B	C	D	Avg
FCFS	0	5	7	6	4.5
SJF	0	8	4	3	3.75

(3)

turnaround time	A	B	C	D	Avg
FCFS	6	8	8	8	7.5
SJF	6	11	5	5	6.75

(4)

waiting time	A	B	C	D	Avg
FCFS	0	5	7	6	4.5
SJF	0	8	4	3	3.75

(5) (already finished)

(6) because SJF is a non-preemptive scheduling strategy; if we want to preempt A, we may use STCF instead

Question 2 - STCF and Round Robin

QUESTION

Use the same workload.

1. Draw the STCF schedule.
2. Calculate response, turnaround, and waiting time under STCF.
3. Draw the Round Robin schedule with (q=2).
4. If a job arrives exactly when a quantum expires, enqueue the new arrival before re-enqueuing the preempted job.
5. Calculate all three metrics under Round Robin.
6. Compare FCFS, STCF, and Round Robin. Which produces the best average turnaround? Which gives the most regular opportunities to run?

ANSWER

Parts (1)-(4).

```
STCF:
Time: 0   1   2   3     5     7           12
Job : | A | B | C |  B  |  D  |     A     |

RR(q=2):
Time: 0     2     4   5     7     9  10    12
Job : |  A  |  B  | C |  A  |  D  | B |  A  |
```

Parts (2)-(5).

response time	A	B	C	D	Avg
STCF	0	0	0	1	0.25
RR	0	1	2	3	1.5

turnaround time	A	B	C	D	Avg
STCF	12	4	1	3	5
RR	12	9	3	5	7.25

waiting time	A	B	C	D	Avg
STCF	6	1	0	1	2
RR	6	6	2	3	4.25

At time 4, the ready queue already contains `C, A`. The newly arriving `D` is appended before the preempted `B` is re-enqueued, producing `C, A, D, B`; `D` does not bypass the already-waiting `A`.

(6) STCF has the best average turnaround under the standard assumptions of known remaining times and negligible preemption overhead. Round Robin gives jobs the most regular opportunities to run.

Question 3 - Context-switch overhead

QUESTION

Ten CPU-bound threads are continuously runnable. Round Robin is used, and a newly ready thread is placed behind the other nine threads.

Each context switch costs (0.1) ms.

1. For (q=1) ms, calculate approximate CPU efficiency.
2. For (q=10) ms, calculate approximate CPU efficiency.
3. For each quantum, calculate approximately how long the newly ready thread may wait before first running.
4. Which quantum provides better response time?
5. Which provides better CPU utilization?
6. Why can neither metric alone determine the best quantum?

For part 3, assume each of the nine preceding threads uses its entire quantum and one context switch follows each quantum.

Review clarification: For the wait estimate, one of the original ten threads is already running and may be partway through its quantum; nine are queued ahead of the newly ready thread. State any assumption about the current thread's remaining quantum.

ANSWER

(1) efficiency ~ 90.91%

(2) efficiency ~ 99.01%

(3) The numerical result depends on when the new thread arrives and on whether the ten threads include it.

Under the exercise's intended interpretation, there are ten threads in total, including the newly ready thread. It arrives at a scheduling boundary with exactly nine complete quanta and nine context switches ahead of it:

```
wait = 9(q + c)
q = 1 ms:   wait = 9.9 ms
q = 10 ms:  wait = 90.9 ms
```

Under the alternative interpretation in which ten threads already exist-one currently running and nine queued-and then an eleventh thread becomes ready, let `r` be the current thread's remaining quantum, where `0 <= r <= q`. The new thread waits through that remainder, nine complete quanta, and ten context switches before executing:

```
wait = r + 9q + 10c
q = 1 ms:   wait ranges from 10.0 to 11.0 ms
q = 10 ms:  wait ranges from 91.0 to 101.0 ms
```

Thus, the original ranges correctly recognized the residual-quantum issue but omitted one context-switch cost under the second interpretation. The exercise wording should have specified the arrival point and whether the newly ready thread was included in the total of ten.

(4) q = 1ms provides better response time

(5) q = 10ms provides better CPU utilization

(6) we must tradeoff between response time and CPU utilization; this vastly depends on the type, length of jobs' burst time, etc.

Question 4 - CPU/I/O convoy effect

QUESTION

At time zero, two jobs are ready:

```
A: CPU 1 → I/O 4 → CPU 1 → I/O 4 → CPU 1
B: CPU 12
```

Assume A is selected first.

1. Draw the execution under non-preemptive FCFS.
2. At what time does A complete?
3. Identify the periods during which the CPU is idle.
4. Identify the periods during which the I/O device is unnecessarily idle because A is ready but cannot run.
5. Explain how a preemptive scheduler with a short quantum improves CPU/I/O overlap.
6. Does improving A’s response necessarily improve B’s completion time?

ANSWER

Use compact half-open time intervals `[start, end)` to represent the schedule.

```
Non-preemptive FCFS

CPU:     [0,1) A | [1,13) B | [13,14) A | [14,18) idle | [18,19) A
A's I/O: [1,5)   |                        [14,18)
```

(1) The CPU schedule is shown above.

(2) A completes at time 19.

(3) The CPU is idle during `[14,18)`.

(4) A becomes ready at time 5 but cannot run until time 13. Thus, the I/O device is unnecessarily idle while A waits during `[5,13)`; A cannot issue its next I/O request until time 14.

(5) For example, Round Robin with `q = 4` produces:

```
CPU:     [0,1) A | [1,5) B | [5,6) A | [6,10) B | [10,11) A | [11,15) B
A's I/O: [1,5)   |           [6,10)
```

The preemptive scheduler lets B use the CPU while A waits for I/O and lets A run promptly after each I/O completion. This improves CPU/I/O overlap and reduces A's post-I/O scheduling latency, ready-queue waiting time, and turnaround time. A's formal response time remains zero under both policies because A first runs at time zero.

(6) Improving A's latency does not necessarily improve B's completion time. Under FCFS, B completes at time 13; under this Round Robin schedule, A completes earlier at time 11, but B completes later at time 15.

(please help me refine the format of my answers; the questions are a bit time-consuming while not challenging)

Question 5 - Conceptual checks

QUESTION

State whether each claim is true or false and explain.

1. Round Robin with an infinitely large quantum behaves like FCFS.
2. Making the quantum arbitrarily small always improves the system.
3. SJF can starve a long job if short jobs continually arrive.
4. STCF must know or estimate each job's remaining CPU time.
5. A work-conserving scheduler can leave the CPU idle while a thread is ready.
6. A scheduler with the lowest average turnaround must also have the lowest response time for every job.
7. A timer interrupt always causes a context switch.
8. A non-preemptive scheduler can still switch jobs when the running job blocks for I/O.

ANSWER

(1) True. With an infinitely large quantum, a CPU-bound job never exhausts its time slice, so no quantum-driven preemption occurs and Round Robin behaves like FCFS.

(2) False. Context switches consume time. As the quantum approaches zero while context-switch cost remains nonzero, an increasingly large fraction of CPU time is spent switching rather than performing useful work.

(3) True. Because SJF is non-preemptive, a waiting long job may starve if a continuous stream of shorter jobs is available whenever the scheduler makes a decision.

(4) True. STCF requires exact knowledge or an estimate of each job's remaining CPU time.

(5) False by definition. A work-conserving scheduler never leaves the CPU idle while any thread is ready. This term was introduced in this lesson and may not appear in the course recording.

(6) False. Minimizing average turnaround does not guarantee the lowest response time for every individual job. For example, STCF may delay the first execution of a long job while Round Robin gives it an earlier time slice, even though STCF has better average turnaround.

(7) False. A timer interrupt transfers control to the kernel, but the scheduler may decide to resume the same thread. Therefore, a timer interrupt does not necessarily cause a context switch.

(8) True. A non-preemptive scheduler may select another job when the running job blocks for I/O, exits, or voluntarily yields.

Lesson 07 - Priority, MLFQ, and multiprocessor scheduling

Question 1 - Transitive priority donation

QUESTION

Assume:

```
L: base priority 1; holds lock A
M: base priority 5; holds lock B, then blocks trying to acquire A
H: base priority 10; blocks trying to acquire B
R: base priority 7; ready and unrelated to either lock
```

1. Without priority donation, which ready thread runs while L and M are in this state?
2. Explain how R indirectly delays H.
3. With transitive donation, calculate the effective priorities of L, M, H, and R.
4. Which thread should run?
5. After L releases A, what happens to L's and M's effective priorities?
6. After M releases B, what happens to M's priority?

7. Explain why simply resetting a thread to its base priority whenever it releases any lock is incorrect.
8. Would priority donation resolve the situation if L waited for B while M waited for A?

ANSWER

(1) without donation, thread R runs

(2) R runs because its priority is higher than L's. This delays L from releasing A, which delays M from acquiring A and later releasing B, which ultimately delays H. Thus, unrelated thread R indirectly delays H.

(3) L: 10, M: 10, H: 10, R: 7

(4) with transitive donation, L should run

(5) L: 10 -> 1. M becomes ready and retains effective priority 10 because H still waits for B, so M should run next.

(6) After M releases B, M: 10 -> 5, and H becomes ready at priority 10.

(7) A thread may still hold another lock on which a higher-priority thread is waiting. Therefore, releasing one lock should remove only the donations associated with that lock and then recompute the thread's effective priority from the remaining donations.

(8) No. If L waits for B while M waits for A, they form a circular wait and deadlock. Priority donation can mitigate priority inversion but cannot break a deadlock cycle.

Question 2 - Multiple donors

QUESTION

Thread T has base priority 2 and holds locks X and Y:

```
A, priority 9, waits for X
B, priority 6, waits for Y
C, priority 4, waits for Y
```

1. What is T's effective priority?
2. T releases X. What should its new effective priority be?
3. T then releases Y. What should its effective priority be?
4. What information must the lock or scheduler retain to calculate these values correctly?
5. Identify the bug in this release implementation:

```
void release(Lock *lock) {
    lock->owner = NULL;
    current->effective_priority = current->base_priority;
    wake_one(lock);
}
```

6. Give a corrected high-level policy; exact code is unnecessary.

ANSWER

(1) 9

(2) 6

(3) 2

(4) The scheduler needs T's base priority, the locks T currently holds, which threads are waiting on each lock, those waiters' effective priorities, and the wait-for relationships required to propagate transitive donation.

(5) The implementation incorrectly discards every donation when it releases any one lock. T may still hold other locks on which higher-priority threads are waiting, so resetting immediately to the base priority may recreate priority inversion.

(6) After releasing a lock:

(1) Remove the donations associated with that released lock.

(2) Recompute `effective_priority` as the maximum of the base priority and the effective priorities donated through all remaining held locks.

(3) Propagate any priority change through the wait-for chain if T is itself blocked on another lock.

(4) Reschedule if the resulting effective priorities require preemption.

Question 3 - Debugging MLFQ

QUESTION

A scheduler uses these rules:

```
New jobs enter the highest queue.
A job is demoted only if it consumes one uninterrupted full quantum.
If it yields early, its high-priority allotment is reset.
There are no periodic priority boosts.
```

1. Give an execution pattern that allows a CPU-bound program to remain at the highest priority indefinitely.
2. Why should accounting use cumulative CPU consumption at a level instead of only uninterrupted execution?
3. Give a workload in which a legitimate CPU-bound thread can starve.
4. Explain how periodic priority boosts address both starvation and changes in workload behavior.
5. Why can excessively frequent boosts damage interactive performance?
6. Does a corrected MLFQ become equivalent to STCF? Explain.

ANSWER

(1) assume quantum = q my 'malicious' program:

```
loop:
    run for q - epsilon
    yield
```

(2) because if not, specially designed programs can remain at the highest priority indefinitely(game the scheduler) and possibly starving all other threads

(3) continous flow of interactive programs which have high priority and a CPU-bound program with low priority. CPU is always busy dealing with interactive programs.

(4) periodic priority boosts will periodically redistribute task priorities, so a CPU-bound task will periodically have high priority to run, which address starvation. If its behavior changes (from CPU-bound to I/O bound), after the boost, the thread will remain in high priority because it only uses CPU shortly and frequently waits for I/O operations.

(5) because frequent boosts will cause CPU-bound programs to have high priorty frequently. CPU will be busy serving high priority CPU-bound programs who does not require and deserve such high priorties.

(6) No. MLFQ is a history-based heuristic that tries to approximate short-job scheduling, whereas STCF minimizes average turnaround time (also called average flow time) under the standard assumptions of known remaining times and negligible preemption overhead. STCF is not being claimed optimal for throughput. MLFQ cannot know future CPU bursts, past behavior may not predict a task's next phase, and priority boosts deliberately trade some turnaround efficiency for adaptation and starvation prevention. Therefore, even a corrected MLFQ is not equivalent to STCF.

Question 4 - Multiprocessor policy review

QUESTION

A two-CPU system uses per-CPU run queues:

```
CPU 0: A, B, C, D
CPU 1: empty
```

A’s working set is hot in CPU 0’s cache. All four threads are CPU-bound.

1. Why is leaving all four threads on CPU 0 undesirable?
2. Why might immediately migrating A specifically be a poor choice?
3. Propose a better migration decision.
4. State the trade-off between load balance and processor affinity.
5. Compare this design with using one global run queue.

6. Suppose H, a very high-priority thread, becomes ready on CPU 0 while both CPUs are running lower-priority threads. What additional action may be necessary to run H promptly on CPU 1?
7. Explain why “never migrate a thread with warm cache state” is not a sound scheduling rule.

ANSWER

(1) Leaving all four tasks on CPU 0 wastes CPU 1 and reduces system throughput. Running some of the work on CPU 1 allows true parallel execution. The idle processor is CPU 1, not CPU 2.

(2) Because A's working set is hot in CPU 0's cache, migrating A may discard useful locality and incur cache-warming and possibly TLB or NUMA penalties.

(3) Leave A on CPU 0 and migrate one of B, C, or D, preferably the task with the lowest estimated migration and locality cost. The prompt does not establish that C or D has a cold working set, so the scheduler should not assume that. It may later rebalance toward two runnable tasks per CPU.

(4) Less migration preserves cache locality and processor affinity but may produce load imbalance, leaving one CPU overloaded while another is idle. More migration improves balance and parallelism but may increase cache, TLB, and NUMA-locality costs.

(5) A global run queue simplifies load balancing and makes it easy to find globally high-priority work, but all CPUs may contend for the shared queue and its lock, limiting scalability. It can still track affinity, although preserving locality is harder and migrations may be more frequent. Per-CPU queues scale better and preserve locality more naturally, but require explicit load balancing.

(6) CPU 0 must make H available to CPU 1-for example, by moving or enqueuing H on CPU 1's run queue-and send CPU 1 a rescheduling interprocessor interrupt (IPI). CPU 1 then enters the kernel, preempts its lower-priority thread, and dispatches H. Merely deciding that CPU 1 should preempt is insufficient; the remote CPU must be notified.

(7) The rule is too absolute: migration is a cost-benefit decision, not an absolute prohibition. We should avoid migrating warm-cache threads when possible, but not never migrate them. cache cost can be smaller than overload / latency cost.

Lesson 08 - Deadlock model and graphs

Question 1 - Runtime graph versus potential graph

QUESTION

Three code paths use single-instance locks:

```
T1: acquire(A); acquire(B);
T2: acquire(B); acquire(C);
T3: acquire(C); acquire(A);
```

Assume each thread has acquired its first lock and is now blocked requesting its second.

1. Write the assignment and request edges in the resource-allocation graph.
2. Write the wait-for graph.
3. Identify the deadlocked set.
4. Show how all four Coffman conditions hold.
5. If T4 continues independent computation, is the entire system deadlocked? State your answer precisely.
6. Before these threads execute, static analysis discovers the acquisition-order cycle A → B → C → A. What does that prove, and what does it not prove?
7. If T2’s second acquisition uses a timeout and releases B after expiration, is the resulting wait permanent deadlock? Which assumption changes?

ANSWER

(1)

```
A -> T1, T1 -> B, B -> T2, T2 -> C, C -> T3, T3 -> A
```

(2)

```
T1 -> T2 -> T3 -> T1
```

(3) deadlock set = {T1, T2, T3}

(4)

- (1) mutual exclusion: lock A, B C can not be used simultaneously
- (2) hold and wait: T1, T2, T3 all holds a lock and wait for another
- (3) no preemption: locks can not be taken away; its owner must release it voluntarily.
- (4) circular wait: easy to know from the runtime wait-for graph

(5) No. Only the deadlock set is deadlocked. T4 will be running without problems.

(6) It proves that a deadlock might take place, it doesn't prove that a deadlock will actually emerge.

(7) This is not a permanent deadlock under the timeout protocol. Before the timeout, the circular wait exists, but the timer is an external event that does not depend on a thread in the cycle. When the timeout expires, T2 cancels its request and voluntarily releases B, breaking hold-and-wait and circular wait. This does not necessarily make locks preemptible; rather, the request is no longer an unconditional indefinite wait.

Question 2 - Multiple instances and misleading cycles

QUESTION

Resources:

```
R: two instances
S: one instance
```

Current state:

```
P1 holds one R and requests S
P2 holds S and requests R
P3 holds the other R and is currently running without requesting anything
```

1. Draw or describe the resource-allocation graph.
2. Identify a cycle.
3. Explain why the cycle does not prove deadlock.
4. Give a completion sequence that allows all processes to finish.
5. Now modify the state so that P3 also requests S. Is the set deadlocked?
6. Explain exactly why the instance released in your answer to part 4 is no longer available.
7. Why would collapsing R into a simple wait-for edge lose important information?

ANSWER

(1)

```
P1 -> S -> P2 -> R -> P3
^                |
|________________|
```

(2) P1 S P2 R forms a cycle

(3) Because R has two instances, P3 can eventually finish and release its instance. P2-not P1-can then acquire that R instance, finish, and release S for P1. Therefore, the cycle does not prove deadlock.

(4) P3 finishes -> one instance of R is released -> P2 acquires R and finishes -> P2 releases S -> P1 acquires S and finishes.

(5) Yes because P1 P2 P3 all hold a resource and wait for a resource owned by another blocked process

(6) Since P3 is blocked (waiting for S), it will never finish and release R.

(7) P2 needs any one instance of R. Initially, the two instances are assigned to P1 and P3, and either holder could potentially release one. A simple wait-for graph might create both `P2 -> P1` and `P2 -> P3`, but these are alternative dependencies rather than two resources that P2 must obtain together. Collapsing the resource node loses the instance count and this OR-wait relationship, so a cycle alone may misleadingly appear to prove deadlock.

Question 3 - Callback and join deadlock

QUESTION

Consider:

```
Lock L;

void parent(void) {
    acquire(&L);

    Thread child = create_thread(worker);
    join(child);

    release(&L);
}

void worker(void) {
    acquire(&L);
    update_shared_state();
    release(&L);
}
```

1. Construct the wait-for relationships.
2. The child does not hold a resource needed by the parent in the usual lock sense. Why is there nevertheless a cycle?
3. Express the dependencies using events rather than only locks.
4. Identify the Coffman conditions in this broader interpretation.
5. Would changing L to a recursive mutex fix the problem?
6. Give two valid redesigns.
7. State a general rule about calling join, waiting on futures, or invoking callbacks while holding locks.

ANSWER

(1)

```
Parent -> Child    // Parent waits for Child to terminate
Child  -> Parent   // Child waits for L, which Parent holds
```

(2) This is not self-deadlock. Parent waits for the child's termination event, while Child waits for Parent to release L. Although Child does not hold a lock required by Parent, completion of Child is itself the event that Parent requires.

(3)

```
Parent holds L
-> Parent waits for child_exit
-> child_exit requires worker to finish
-> worker must acquire L
-> L cannot be released until join(child) returns
```

(4) The four conditions are present in the generalized event-dependency model:

(1) Mutual exclusion: L has only one owner.

(2) Hold and wait: Parent holds L while waiting for Child to terminate.

(3) No preemption: L cannot be forcibly taken from Parent.

(4) Circular wait: Parent waits for Child, while Child waits for Parent to release L.

(5) No. A recursive mutex permits the same thread to reacquire its own lock, but `parent()` and `worker()` execute in different threads.

(6) A simple redesign is to release L before `join` and reacquire it afterward only if Parent needs to inspect or modify protected state:

```
Lock L;
void parent(void) {
    acquire(&L);
    prepare_shared_state();
    release(&L);

    Thread child = create_thread(worker);
    join(child);
```

```
    acquire(&L);
    consume_shared_state();
    release(&L);
}

void worker(void) {
    acquire(&L);
    update_shared_state();
    release(&L);
}
```

Another redesign uses a condition variable with an explicit predicate. The condition-variable wait releases L while Parent sleeps, and `join` occurs after L has been released:

```
Lock L;
Condition cv;
bool done = false;

void parent(void) {
    acquire(&L);

    Thread child = create_thread(worker);

    while (!done) {
        wait(&cv, &L);
    }

    release(&L);
    join(child);
}

void worker(void) {
    acquire(&L);

    update_shared_state();
    done = true;
    signal(&cv);

    release(&L);
}
```

(7) Never hold a lock while invoking a callback, joining a thread, or waiting for a future if completion may require code that acquires that lock. Release the lock first, or wait through a condition-variable operation that atomically releases it.

Question 4 - Deadlock-free but still broken

QUESTION

Every philosopher uses:

```
while (true) {
    acquire(&left);

    if (try_acquire(&right)) {
        eat();
        release(&right);
        release(&left);
        return;
    }

    release(&left);
}
```

1. Explain why the original blocking circular-wait deadlock has been removed.
2. Construct a livelock execution.
3. Which Coffman condition is prevented from persisting?
4. Does random backoff prove that livelock is impossible?
5. Can an individual philosopher still starve even if other philosophers eat?
6. Propose one policy that provides a stronger progress guarantee.

7. State separately which safety and liveness properties your policy aims to guarantee.

ANSWER

(1) `try_acquire(right)` never blocks while the philosopher retains `left`. On failure, the philosopher releases `left`, so hold-and-wait and the persistent blocking circular wait are removed.

(2) every philosopher's action is synchronized (take left → fail right → release left → retry)

(3) 'hold and wait'

(4) No. Random backoff reduces the probability of livelock, not deadlock. It does not provide a deterministic proof that livelock is impossible, although independent random delays may make infinite livelock have probability zero under suitable assumptions.

(5) yes. a philosopher may starve if everytime he tries, the one on his right is always eating

(6) A decentralized solution with identical code gives every fork a unique, globally comparable ID and requires every philosopher to acquire the lower-ID fork before the higher-ID fork:

```
void dine(Fork *left, Fork *right) {
    Fork *first  = left->id < right->id ? left : right;
    Fork *second = left->id < right->id ? right : left;

    acquire(first);
    acquire(second);
    eat();
    release(second);
    release(first);
}
```

No central waiter and no philosopher-specific branch are required. The code is identical for every philosopher, but the unique fork IDs provide the symmetry-breaking information.

(7) Safety: every fork is protected by a mutually exclusive lock, so adjacent philosophers cannot eat simultaneously. Deadlock freedom: every nested acquisition follows a strictly increasing fork-ID order. A circular wait would require fork IDs to increase strictly around a cycle and return to the starting ID, which is impossible. The algorithm uses blocking acquisition and therefore does not create the earlier retry livelock. Starvation freedom additionally requires fair locks and fair scheduling; resource ordering alone proves deadlock freedom but not starvation freedom.

Question 5 - Diagnose the progress failure

QUESTION

Classify each scenario as deadlock, livelock, starvation, ordinary blocking, or “insufficient information.” State the assumption on which your answer depends.

1. A thread is blocked waiting for disk I/O.
2. Two threads hold one lock each and wait indefinitely for the other.
3. A low-priority ready thread never runs because higher-priority work continually arrives.
4. Two threads repeatedly release and reacquire locks but never enter their critical sections.
5. All worker threads wait on a condition variable, and no existing thread can ever signal it.
6. All worker threads wait on a condition variable, but an external network request may eventually cause a signal.
7. A resource graph contains a cycle, and one resource type in that cycle has three instances.
8. A thread attempts to acquire a non-recursive mutex that it already owns.

ANSWER

(1) Ordinary blocking, assuming the device and driver eventually complete the request. Without that assumption, an external device failure could leave the thread indefinitely blocked.

(2) Deadlock, assuming both acquisitions block indefinitely and neither thread releases its held lock.

(3) Starvation. Other threads continue making progress while the low-priority ready thread is continually denied the CPU.

(4) Livelock, assuming the threads keep executing and changing lock state but never complete useful work.

(5) Deadlock if progress requires a signal and neither an existing thread nor an external event can produce one. If no work is expected, the same state may instead be intentional quiescence.

(6) Ordinary blocking if the network event eventually occurs and signals the condition variable. Because the prompt says only that it may occur, the unconditional classification is insufficient information.

(7) Insufficient information. A cycle involving a resource type with multiple instances does not establish deadlock without the instance allocation, availability, outstanding requests, and possible completion information.

(8) Self-deadlock, assuming acquisition of an already-owned non-recursive mutex blocks rather than returning an error.

Lesson 09 - Deadlock handling

Question 1 - Which condition is actually broken?

QUESTION

For each policy:

1. Identify the Coffman condition it is intended to eliminate.
2. Explain whether it truly eliminates that condition or merely reduces the probability of deadlock.
3. Give one important cost or limitation.

Policies:

1. Require all locks to be acquired in increasing numerical order.
2. If the second lock is unavailable, release the first and retry after a random delay.
3. Require a process to acquire every resource it may need before starting.
4. Abort a database transaction and restore its checkpoint when it cannot obtain a lock.
5. Replace direct printer ownership with a spooler that accepts immutable print jobs.
6. Use a ten-second timeout for every mutex acquisition.
7. Use try_acquire and immediately report failure to the caller without retaining any previously acquired lock.

ANSWER

(1) "Require all locks to be acquired in increasing numerical order"

- Coffman condition intended to eliminate: circular wait
- Truly eliminates that condition: every wait-for edge follows the same global resource order, so a cycle cannot form.
- Important cost: the system must define and maintain a global order, which can be difficult across subsystems or with dynamically discovered resources. It may also force awkward early acquisition or release/reacquisition. It does not inherently serialize threads; threads using unrelated locks can still run concurrently.

(2) "If the second lock is unavailable, release the first and retry after a random delay"

- Coffman condition intended to eliminate: hold and wait
- Truly eliminates that condition, provided that the thread releases every previously acquired resource before waiting or retrying.
- Limitation: it may cause livelock, starvation, repeated work, and poor resource utilization.

(3) "Require a process to acquire every resource it may need before starting"

- Coffman condition intended to eliminate: hold and wait
- Truly eliminates that condition: a process never waits for additional resources while retaining resources it already holds.
- Limitation: poor resource utilization, long admission delays or starvation, and the requirement that each process know its maximum resource needs in advance.

(4) "Abort a database transaction and restore its checkpoint when it can not obtain a lock"

- Coffman condition intended to eliminate: primarily no preemption. Rollback lets the system reclaim the transaction's locks. Because the transaction aborts instead of waiting while retaining locks, the policy also prevents persistent

hold and wait.

- It eliminates the relevant condition only if aborting immediately releases all locks and other resources held by the transaction.
- Limitation: transactions may repeatedly abort, lose work, or starve; rollback also requires recoverable state.

(5) "Replace direct printer ownership with a spooler that accepts immutable print jobs"

- Coffman condition intended to eliminate: mutual exclusion
- It removes mutual exclusion from the client processes' view: clients submit shareable jobs rather than owning the printer. The physical printer is still exclusively controlled by the spooler.
- Limitation: jobs generally need to be self-contained or immutable, and the spooler introduces storage overhead, a centralized bottleneck, and a possible single point of failure.

(6) "Use a ten-second timeout for every mutex acquisition"

- Coffman condition intended to eliminate: circular wait
- It does not prevent a circular wait from forming. It only limits how long a suspected deadlock persists by eventually cancelling an acquisition or operation.
- Limitation: doesn't eliminate deadlock; cannot distinguish deadlock from slow work; may incorrectly kill valid operations

(7) "Use try_acquire and immediately report failure to the caller without retaining any previously acquired lock"

- Coffman condition intended to eliminate: Hold and wait
- Truly eliminates that condition, provided that failure causes the caller to release all previously acquired locks before returning or retrying. try_acquire alone does not prevent deadlock if earlier locks are retained.
- Limitation: repeated unsuccessful attempts may cause livelock or starvation.

Question 2 - A small Banker problem

QUESTION

There are two resource types, A and B.

Current state:

Process	Allocation	Maximum	Need
P0	(1, 0)	(2, 2)	?
P1	(0, 1)	(2, 2)	?
P2	(1, 1)	(1, 2)	?

```
Available = (1, 1)
```

1. Calculate every Need.
2. Find a safe sequence.
3. P0 requests (0, 1). Show that it satisfies both local checks:

```
Request <= Need
Request <= Available
```

4. Tentatively grant the request and run the safety test.
5. Should Banker’s algorithm grant it?
6. If it were granted, would the system already be deadlocked at that instant?
7. Give a valid future request pattern that could produce deadlock from the resulting unsafe state.
8. Explain why “the requested resources are currently available” is insufficient justification for granting them.

ANSWER

(1)

```
Need:
    P0: (1, 2)
    P1: (2, 1)
```

```
    P2: (0, 1)
```

(2)

```
Safe Sequence:
    P2 -> P0 -> P1
```

(3)

```
P0.need = (1, 2)
Available = (1, 1)
so request = (0, 1) <= P0.need and request <= Available (component-wise)
```

(4) after the tentative allocation:

```
Need:
    P0: (1, 1)
    P1: (2, 1)
    P2: (0, 1)

Available:
    (1, 0)
```

There is no process whose remaining Need is component-wise less than or equal to Available.

(5) No. Banker's algorithm should deny or postpone the request because the tentative allocation produces an unsafe state.

(6) No. It only implies that the system is in an unsafe state. If the processes have not yet issued incompatible outstanding requests, they are not currently deadlocked.

(7) One future request pattern that produces deadlock after the tentative grant is:

```
P0 requests its remaining (1, 1) and blocks.
P1 requests its remaining (2, 1) and blocks.
P2 requests its remaining (0, 1) and blocks.
```

At this point, Available is `(1, 0)`. Every process still needs at least one unit of B, while all units of B are held by processes that are blocked and retain their current allocations. No process can complete and release its resources, so the system is deadlocked.

(8) The fact that the requested resources are currently available does not guarantee that the requesting process can proceed to completion. Safety also depends on whether there is some future completion order in which every process can obtain its remaining maximum claim and release its allocation. An unsafe grant may permit later requests to produce deadlock.

Question 3 - Repair the transfer operation

QUESTION

Given the original `transfer` implementation:

```
void transfer(Account *from, Account *to, int amount) {
    acquire(&from->lock);
    acquire(&to->lock);
    from->balance -= amount;
    to->balance += amount;
    release(&to->lock);
    release(&from->lock);
}
```

1. Construct the two-thread deadlocking execution for transfer(A, B) and transfer(B, A).
2. Repair it using a resource ordering.
3. Explain why comparing raw IDs is insufficient if IDs can collide or change.
4. Explain the from == to case. What failure can occur without the special case?
5. Suppose every account lock follows the order, but an audit function acquires log_lock while holding an account lock, whereas another function acquires an account lock while holding log_lock. Is the program deadlock-free?
6. State the ordering rule needed across both account locks and log_lock.

7. Is releasing the first lock and retrying the operation equivalent to ordering? Compare deadlock, livelock, and starvation guarantees.

ANSWER

(1)

```
// original transfer function (flawed)
void transfer(Account *from, Account *to, int amount) {
    acquire(&from->lock);
    acquire(&to->lock);

    from->balance -= amount;
    to->balance += amount;

    release(&to->lock);
    release(&from->lock);
}
```

Assume that thread P1 executes `transfer(A, B)` while P2 executes `transfer(B, A)`:

```
1. P1: acquire(&A->lock)
2. P2: acquire(&B->lock)
3. P1: acquire(&B->lock)  // blocks while holding A
4. P2: acquire(&A->lock)  // blocks while holding B
```

The threads have formed a circular wait and are deadlocked.

(2)

```
// corrected transfer function
void transfer(Account *from, Account *to, int amount) {
    if (from == to)
        return;

    int from_id = from->id;
    int to_id = to->id;
    Account *first = (from_id < to_id) ? from : to;
    Account *second = (from_id < to_id) ? to : from;

    acquire(&first->lock);
    acquire(&second->lock);

    from->balance -= amount;
    to->balance += amount;

    release(&second->lock);
    release(&first->lock);
}
```

(3) Stable, unique IDs define a global total order, eliminating circular wait. Every thread that needs the same pair of locks agrees on which one must be acquired first. If IDs collide without a stable tie-breaker, the order is not total; if IDs change while the objects participate in locking, different threads may follow inconsistent orders. Either defect can allow circular wait to reappear.

(4) If `from == to`, the function would try to acquire the same ordinary non-recursive mutex twice, causing self-deadlock. Returning early is correct because transferring money from an account to itself is a no-op.

(5) No. Once a thread blocks inside the second `acquire`, it cannot execute ordinary user-level code to release the first lock. Release-and-retry requires a nonblocking operation such as `try_acquire`, a cancellable acquisition, or an external recovery mechanism.

(6) If `log_lock` and account locks are both needed, define and obey one global order on every code path:

```
log_lock < account[smaller ID] < account[larger ID]
```

Release them in reverse order. This eliminates circular wait, although holding the global log lock across every transfer may serialize otherwise independent transfers.

(7) No, deadlock freedom does not imply starvation freedom.

```
lock ordering: eliminates deadlock and does not itself produce livelock, but an unfair lock can still
    cause starvation
release and retry: eliminates deadlock only if failure releases every held lock, but repeated retries may
    cause livelock or starvation
```

Question 4 - Detection and recovery policy

QUESTION

A database detects this cycle:

```
T1 → T2 → T3 → T1
```

Metadata:

Transaction	Age	Work completed	Resources held	Previous rollbacks
T1	100 s	90%	2	0
T2	10 s	5%	1	3
T3	20 s	40%	5	0

1. Give an argument for choosing each transaction as the victim.
2. Which victim would you choose, and what objective are you optimizing?
3. Why is “always abort the transaction with the least work completed” unsafe as a long-term policy?
4. Design a victim score that accounts for both rollback cost and starvation.
5. After aborting one transaction, what must the system do before waking the remaining transactions?
6. Could the same wait-for cycle reappear immediately? Give a scenario.
7. Compare detection-and-recovery with Banker-style avoidance for a database whose transactions do not know their maximum lock requirements in advance.

ANSWER

(1)

- T1 has not previously been rolled back, so choosing it is less likely to be unfair.
- T2 is young, so aborting it may discard relatively little completed work.
- T3 holds many resources, so aborting it may unblock more transactions; it is also still at an early stage of completion.

(2) I would choose transaction T2 if the objective is to minimize immediately discarded work: it is young (`age = 10 s`) and has probably performed relatively little work. Its age does not necessarily mean that the transaction is small, however; its total future runtime could still be large.

(3) Always choosing the transaction with the least completed work may starve it. After every rollback it restarts near zero progress, so the same policy may repeatedly choose it as the cheapest victim.

(4) The original proposal,

```
completion_percentage * (previous_rollbacks + 1)
```

is always zero when completion is zero, regardless of the number of previous rollbacks. A more robust policy is:

```
victim_cost = estimated_work_lost
            + alpha * previous_rollbacks
            - beta  * recovery_benefit
```

Select the transaction with the smallest cost. `recovery_benefit` may estimate the number of resources released or transactions unblocked. The rollback-count penalty protects repeatedly aborted transactions from starvation.

(5) Recovery should proceed in this order:

- (1) Select enough victim transactions to break every detected cycle.
- (2) Abort and roll back each victim while its locks still protect the affected state.
- (3) Release its locks and other resources.
- (4) Wake or re-evaluate waiting transactions.
- (5) Restart the victim later, preferably with an anti-starvation policy.

Rollback must happen before releasing locks; otherwise another transaction could observe inconsistent intermediate state. The system restores transaction or database state, not generally the thread's original execution state.

(6) Yes. For example, initially T1 holds A and waits for B, T2 holds B and waits for C, and T3 holds C and waits for A. The system aborts T2 and releases B. If restarted T2 reacquires B before T1 does and then waits for C, the same cycle immediately reforms. Detection and recovery do not prevent recurrence; delayed restart, increasing victim priority, or randomized backoff can reduce it.

(7) Banker's algorithm requires each process to declare its maximum resource needs in advance. If database transactions cannot state their maximum lock requirements before execution, the safety test cannot be applied reliably, so Banker-style avoidance is unsuitable.

Deadlock detection and recovery does not require advance knowledge of maximum claims. It constructs a wait-for graph from runtime lock requests and grants. When it detects a cycle, it selects a victim, rolls it back, and releases its resources. The trade-off is that deadlocks are permitted to occur, transactions lose work, and the system pays detection and recovery overhead.

Lesson 10 - Address-space foundations

Question 1 - Translation and fault classification

QUESTION

A machine uses 16-bit virtual addresses. The upper two bits select a segment, and the lower fourteen bits are the offset. A process has:

Segment	Base	Bounds	Permissions
0	`0x4000`	`0x1200`	`R-X`
1	`0x9000`	`0x0800`	`RW-`
2	`0xC000`	`0x1000`	`R--`
3	invalid	-	-

The valid-offset rule is `offset < bounds`.

For each access, give the segment, offset, and either the physical address or exact fault reason:

1. Instruction fetch from VA 0x1100
2. Write to VA 0x47F0
3. Read from VA 0x4900
4. Write to VA 0x8200

If one of these accesses faults, does that necessarily cause a context switch? Explain briefly.

ANSWER

(1)

- (1) segment 0, offset 0x1100, physical address 0x5100
- (2) segment 1, offset 0x07F0, physical address 0x97F0
- (3) segment 1, offset 0x0900, bounds fault (offset >= bounds)
- (4) segment 2, offset 0x0200, write-protection fault (the segment is read-only)

Not necessarily. A fault necessarily traps into kernel mode, but the kernel may deliver a signal and later resume the same process. If the default signal action terminates the process, another process must eventually be scheduled, but that context switch is a consequence of termination rather than an inherent part of entering the fault handler. A *bounds fault* or *protection fault* identifies the precise hardware cause; *segmentation fault* commonly describes the eventual Unix signal or process-level outcome.

Question 2 - Find the isolation bug

QUESTION

Initially, process `P` is running:

```
P: base = 0x10000, bounds = 0x3000
Q: base = 0x20000, bounds = 0x2000
```

The kernel context-switches from `P` to `Q` but accidentally leaves `P`'s base and bounds registers installed. `Q` then loads from virtual address `0x0100`.

Answer:

1. Does the bounds check pass?
2. What physical address is accessed?
3. Why is this an isolation violation rather than merely an incorrect translation?
4. State the kernel invariant that prevents this bug.

ANSWER

(1) Yes, it does.

(2) 0x10100

(3) The arithmetic translation is consistent with the installed registers, but those registers belong to the wrong address space. Consequently, Q's load reads P's private physical memory. Later stores could also modify P's memory if no additional permission mechanism prevented them.

(4) Before returning to user mode after every context switch, the kernel must install MMU translation state that belongs to the process designated as currently running. User code must not be able to modify this privileged state.

Question 3 - Fragmentation counterexample

QUESTION

Physical memory currently contains free holes of `100 KiB`, `500 KiB`, and `200 KiB`.

1. Can a 600 KiB process be admitted using one base-and-bounds region? Why?
2. Suppose the process is divided into segments of 400 KiB code, 200 KiB heap, and 100 KiB stack. Can segmentation place it? Give one placement.
3. Does this example prove that segmentation eliminates external fragmentation? Explain.

ANSWER

(1) No. Base-and-bounds requires one contiguous physical region for the process. Every individual hole is smaller than 600 KiB, even though the total free space is 800 KiB. This is external fragmentation.

(2) Yes. Place the 400 KiB code segment inside the 500 KiB hole, leaving 100 KiB there; place the 200 KiB heap in the 200 KiB hole; and place the 100 KiB stack in the original 100 KiB hole.

(3) No. Segmentation does not eliminate external fragmentation because every individual segment must still occupy a contiguous physical region. For example, with free holes of 300 KiB, 300 KiB, and 400 KiB, a process with a 450 KiB code segment, 100 KiB heap, and 100 KiB stack cannot fit: no hole can contain the code segment even though total free memory exceeds total demand.

Question 4 - Protection and sharing design

QUESTION

Processes `P` and `Q` must share immutable library code, but their global variables and heaps must remain private.

Using segmentation, explain:

1. Which segment-table entries may refer to the same physical memory?
2. What permissions should the shared library receive?
3. How the private writable segments remain isolated.

4. Why a single base-and-bounds pair would make this arrangement awkward.

ANSWER

(1) P's and Q's library segment-table entries may point to the same physical code region. Their segment indices and virtual addresses need not be identical.

(2) Read and execute only (`R-X`).

(3) Their writable data/global and heap segments have distinct bases pointing to disjoint physical memory. Their stacks should ordinarily be private as well.

(4) One base-bounds defines exactly one contiguous virtual-to-physical mapping for the whole process address space. You cannot have part of that range mapped to shared library physical memory and another part mapped to private physical memory.

Lesson 11 - Paging and TLBs

Question 1 - Trace the translation path

QUESTION

A 32-bit system uses 4 KiB pages. The TLB initially contains:

VPN	PFN	Permissions
`0x12345`	`0x00A7`	`R-X`

Relevant page-table entries are:

VPN	PFN	Present	Permissions	A	D
`0x22222`	`0x0555`	1	`RW-`	0	0
`0x33333`	-	0	-	0	0
`0x44444`	`0x0777`	1	`R--`	0	0

For each independent access, give the VPN, offset, TLB result, and final physical address or exact fault:

1. Fetch an instruction from 0x12345678.
2. Store to 0x22222ABC.
3. Load from 0x33333010.
4. Store to 0x44444008.

Also give the resulting `A` and `D` bits after each successful access.

ANSWER

(1) `VPN = 0x12345`, `offset = 0x678`: TLB hit. The execute permission allows the instruction fetch, so `PA = 0x00A7678`. The accessed bit becomes `A = 1`, while the dirty bit remains unchanged.

(2) `VPN = 0x22222`, `offset = 0xABC`: TLB miss. The PTE is present and writable, so `PA = 0x0555ABC`. After the successful store, `A = 1` and `D = 1`, and the translation can be inserted into the TLB.

(3) `VPN = 0x33333`, `offset = 0x010`: TLB miss followed by a non-present PTE, so the processor raises a page fault. The kernel must determine whether this is a valid nonresident page or an invalid access.

(4) `VPN = 0x44444`, `offset = 0x008`: TLB miss. The PTE is present but read-only, so the store raises a write-protection fault. `0x0777008` is the candidate translated address, but no physical-memory write occurs.

Question 2 - Context-switch isolation failure

QUESTION

An untagged TLB contains an entry left by process P:

```
VPN 0x42 → PFN 0x10, RW
```

Process Q's page table instead contains:

```
VPN 0x42 → PFN 0x90, RW, present
```

The kernel switches from P to Q, changes the page-table base register, but forgets to flush the TLB. Q loads from `VA 0x00042008`.

Explain:

1. Which physical address is actually accessed.
2. Why changing the page-table base did not prevent the bug.
3. Two valid ways to prevent it.
4. If the TLB had been flushed, whether Q's access would necessarily cause a page fault or enter the kernel.

ANSWER

(1) The stale entry maps `VPN 0x42` to `PFN 0x10`. With offset `0x008`, the physical address actually accessed is `0x00010008`. The intended address under Q's page table would have been `0x00090008`.

(2) On a TLB hit, the processor uses the cached PFN and permissions without walking the current page table. Changing the page-table base therefore does not affect P's stale cached translation.

(3) Two valid solutions are to flush the relevant untagged TLB entries during the context switch, or to tag entries with address-space identifiers (ASIDs) and install Q's ASID. An ASID identifies an address space; threads sharing one process usually share the same ASID.

(4) No. After a flush, Q's access produces a TLB miss, but Q's PTE is present. A hardware-managed TLB can walk the page table and refill the entry without entering the kernel. A software-managed TLB may trap to the kernel on the miss. Neither case is a page fault.

Question 3 - Stale permissions

QUESTION

A TLB contains:

```
VPN 0x77 → PFN 0x300, RW
```

The kernel changes the corresponding PTE from `RW` to `R--` but does not invalidate the TLB. The process then stores to `VA 0x00077020`.

Explain what may happen, why consulting the modified PTE is not guaranteed, and what the kernel must do on both a uniprocessor and a multicore system.

ANSWER

The stale TLB entry still grants write permission, so the store may succeed at `PA = 0x00300020` even though the current PTE is read-only. A TLB hit supplies both the PFN and cached permissions, so the processor is not guaranteed to consult the modified PTE.

On a uniprocessor, the kernel must update the PTE and invalidate the corresponding local TLB entry. On a multicore system, it must also perform a TLB shootdown on every core that might have cached this address-space translation. The kernel may track those cores instead of broadcasting to literally every processor, but it must receive the necessary invalidation acknowledgements before treating the permission revocation as effective.

Question 4 - Page-size trade-off

QUESTION

A 32-bit machine uses flat page tables, 4-byte PTEs, and a 64-entry TLB.

Compare 4 KiB pages with 64 KiB pages:

1. Calculate the flat page-table size under each choice.
2. Calculate the TLB reach under each choice.

3. Explain two advantages and two disadvantages of choosing 64 KiB pages.

ANSWER

(1)

```
4 KiB pages:  12-bit offset, 20-bit VPN, 2^20 PTEs
                page-table size = 4 * 2^20 bytes = 4 MiB

64 KiB pages: 16-bit offset, 16-bit VPN, 2^16 PTEs
                page-table size = 4 * 2^16 bytes = 256 KiB
```

(2)

```
4 KiB pages:  64 * 4 KiB  = 256 KiB of TLB reach
64 KiB pages: 64 * 64 KiB = 4 MiB of TLB reach
```

(3) Advantages of 64 KiB pages include greater TLB reach, potentially improving the hit rate; smaller page tables; and fewer compulsory page faults for workloads with spatial locality because each fault maps more nearby data.

Disadvantages include greater internal fragmentation and coarser-grained I/O. A fault or eviction transfers a larger unit and may move unused data. A single write also marks the entire page dirty at page granularity, so its later eviction may require writing back the whole 64 KiB page.

Lesson 12 - Scalable page tables

Question 1 - Multilevel translation and memory cost

QUESTION

A 32-bit machine uses:

- 4 KiB pages
- 4-byte PDEs and PTEs
- A two-level 10 | 10 | 12 address split
- A page directory at physical address 0x00100000

A process maps only these eight pages:

```
six pages: 0x00400000-0x00405FFF
one page:  0x007FF000
one page:  0x00800000
```

The relevant structures include:

```
PDE[1]    → second-level table at PA 0x00200000
PDE[2]    → second-level table at PA 0x00300000
PTE[1][1023] → PFN 0x345, present, R--
PTE[2][0]    → PFN 0x456, present, RW-
```

Answer the following:

1. Split VA 0x007FFABC and VA 0x00800020 into directory index, table index, and offset.
2. How many second-level tables are required for the eight mappings? Explain why two adjacent virtual pages can require different tables.
3. How much memory do the page-table structures consume, including the directory but excluding mapped data pages?
4. For a load from VA 0x007FFABC, calculate:
   - the physical address of its PDE;
   - the physical address of its PTE;
   - the final physical address.
5. Assuming a cold TLB and no caching of page-table data, how many physical-memory accesses does that load require?
6. Compare this process's multilevel page-table memory cost with a flat page table.

ANSWER

(1) VA 0x007FFABC: directory idx = 0x001; table idx = 0x3FF; offset = 0xABC; VA 0x00800020: directory idx = 0x002; table idx = 0x000; offset = 0x020;

(2) Two second-level tables are required. Each table covers one aligned 4 MiB virtual region. `0x007FF000` is the final page under `PDE[1]`, while the adjacent page at `0x00800000` is the first page under `PDE[2]`.

(3) Each page-table structure occupies one 4 KiB page:

```
one page directory      =  4 KiB
two second-level tables =  8 KiB
total                   = 12 KiB
```

(4) VA 0x007FFABC PA of PDE = 0x00100000 + 4 * 0x001 = 0x00100004 PA of PTE = PDE[1] + 4 * 0x3FF = 0x00200FFC final PA = 0x00345ABC

(5) visit PDE -> visit PTE -> visit PA, 3 physical memory access.

(6) A flat page table with 4 KiB pages has `2^20` entries and consumes 4 MiB. On a TLB miss with no caching, it needs one PTE access plus the requested data access, compared with three accesses for this two-level design. The multilevel table therefore trades additional walk latency for a much smaller page-table footprint here: 12 KiB rather than 4 MiB.

Question 2 - Shootdown and ASID adversarial execution

QUESTION

Cores C0 and C1 are running two threads of process P. Both cache:

```
(ASID 7, VPN 0x50) → PFN 0x100, RW
```

C0 changes P's PTE for `VPN 0x50` to non-present, invalidates only its own TLB, and immediately gives `PFN 0x100` to process Q. C1 then stores through P's old virtual address.

1. Trace what C1 does and explain exactly how Q's memory can be corrupted.
2. Why will C1 not necessarily notice that P's PTE is now non-present?
3. Give a correctly ordered shootdown and frame-reclamation protocol.
4. Must the kernel wait for acknowledgements before reusing the frame? Why?
5. If the kernel merely changes the PTE from RW to R-- without reusing the frame, is a shootdown still required?
6. Does switching from one P thread to the other inherently require a flush?
7. Suppose ASID 7 is later reassigned to an unrelated process R without invalidation. Explain the resulting risk and give one safe ASID-reuse policy.

ANSWER

(1) when C1 stores through P's old virtual address(specifically VPN 0x50), it will result in a TLB hit and will not check the page tables; so even though the physical page (PFN 0x100) has been reassigned to process Q and is invalid for process P, C1 (running process P) can still read or write to memory area private to process Q, potentially corrupting Q's memory.

(2) because the entry is cached in TLB, and if TLB hits, it will not look up the PTE.

(3)

- (1) Synchronize the address-space modification.
- (2) Modify the PTE to mark it non-present, preventing new page-table walks from recreating the old translation.
- (3) Perform the memory-ordering operation required by the architecture.
- (4) Invalidate the corresponding entry in C0's local TLB.
- (5) Send shootdown requests to every other core that might have cached the translation, not merely those currently running P.
- (6) Wait for acknowledgements from all target cores.
- (7) Only after the acknowledgements arrive may the kernel reclaim and reassign the physical frame.

(4) Yes. without waiting, some cores may still have the outdated TLB entry and writes to it even after the page is relocated to another process. after all acknowledgements arrive, no TLB is carrying corresponding entries, so it is safe

now to reassign them.

(5) Yes, it is stilled required because cores may hold stale TLB entries with old writable permission

(6) No. Both threads belong to the same process, share the same page table and same ASID. The TLB entries remain valid for both threads. No flush is necessary.

(7) Old stale TLB entries tagged with ASID 7 from process P still linger in TLB. When process R gets assigned ASID 7, R's virtual addresses may collide with VPN values previously used by P. The hardware can match on `(ASID,VPN)` and use the old P-era TLB translation for R's accesses. R gets wrong physical mappings; memory corruption, information leak, privilege violation occur.

One safe reuse policy is to invalidate all TLB entries tagged with ASID 7 on every relevant core and wait for completion before assigning that ASID to R. Alternatively, the kernel can use ASID generations and perform an appropriate flush when the finite ASID namespace wraps.

Question 3 - Huge and shared mapping design

QUESTION

Eight inference processes share a `128 MiB` immutable model-weight region. The machine supports 4 KiB and 2 MiB pages. Assume a TLB can hold 64 entries of the selected page size.

Each process also has a private 1 MiB heap that is sparsely modified.

1. Calculate the number of pages needed for the shared weight region under each page size.
2. Calculate the corresponding TLB reach. Can 64 entries cover the entire weight region?
3. Ignoring page-table overhead, how much physical memory should the shared weights consume across all eight processes?
4. Do the processes still need separate virtual mappings and ASID-tagged TLB entries even though the PFNs are shared?
5. What permissions should the model-weight mappings receive?
6. Explain why 2 MiB pages are attractive for the weight region.
7. Explain why 2 MiB pages may be a poor choice for each sparse private heap.
8. If the weights were initially copy-on-write and one process wrote one byte, what could happen with a huge-page mapping? Describe two implementation choices.
9. When one inference process exits, may the kernel immediately free all shared weight frames? Explain.

ANSWER

(1) 4 KiB pages: 32,768 pages; 2 MiB pages: 64 pages.

(2) The 4 KiB entries provide 256 KiB of TLB reach and cannot cover the region. The 2 MiB entries provide 128 MiB of reach and cover the region exactly, assuming all 64 entries are available and ignoring associativity conflicts.

(3) 128 MiB in either case, excluding page-table overhead.

(4) Yes. Each process has its own virtual address space; each needs its own PTE mapping its virtual-page number to the common PFN. TLB entries are tagged by ASID, so different-ASID processes can hold separate TLB entries pointing to identical physical frames.

(5) `R--`: readable, non-writable, and non-executable because the weights are immutable data.

(6) Sixty-four 2 MiB TLB entries cover the 128 MiB weight region exactly under the simplified assumptions, greatly reducing TLB misses after warm-up. Huge pages also reduce the number of leaf mappings from 32,768 to 64 and can shorten page-table walks.

(7) Each huge page is 2 MiB while the heap is only 1 MiB, causing about 1 MiB of internal fragmentation per process. Huge pages also require larger aligned physical allocations and make faults, copy-on-write, dirty tracking, protection changes, and reclamation more coarse-grained-poor matches for a sparse, lightly modified heap.

(8) Huge-page copy-on-write: writing a single byte triggers copy-on-write on the huge page. Choice 1: copy the entire 2 MiB huge page. The writer gets a private 2 MiB copy; other processes keep mapping the original huge page. Drawback: high memory overhead for a single-byte write. Choice 2: split the writer's huge-page mapping into base 4 KiB PTEs, copy

only the modified 4 KiB subpage, and keep its other subpages mapped to the original frames. Other processes may retain the original huge mapping. Changing the mapping also requires invalidating the writer's old huge-page TLB entry.

(9) No. Reference counting or equivalent ownership tracking is required. The other seven processes still have active mappings, so the kernel only removes the exiting process's mappings and decrements the relevant reference counts. A shared frame may be freed only after its final mapping and any other kernel references disappear.

Lesson 13 - Demand paging

Question 1 - Classify and handle page faults

QUESTION

Assume free frames are initially available. File and swap contents mentioned below are not already cached.

A process has these regions:

Region	State
`0x00400000–0x00403FFF`	Anonymous, `RW-`, valid, nonresident, never allocated
`0x00500000–0x00500FFF`	File-backed, `R--`, valid, nonresident
`0x00600000–0x00600FFF`	Anonymous, `RW-`, valid, nonresident, contents in swap slot 17
`0x00700000–0x00700FFF`	Present at PFN `0x90`, `R-X`
All other addresses	Unmapped

Treat the following accesses independently:

1. Store to 0x00401234.
2. Load from 0x00500100.
3. Load from 0x00600020.
4. Store to 0x00700008.
5. Load from 0x00800000.

For each access:

- Classify the fault or successful access precisely.
- State whether storage I/O is required.
- State whether the thread must block under the assumptions.
- Describe the essential PTE or metadata change.
- State whether the instruction can be restarted.
- For the demand-zero store, explain when the page becomes dirty.

Finally, explain why “PTE not present” is insufficient information for deciding whether an access is legal.

ANSWER

(1) Store to 0x00401234

- This is a demand-zero page fault because the page is valid but has never been allocated.
- Storage I/O is not required because a free frame is available. The thread need not enter the blocked state: the handler can synchronously allocate and zero a frame while running in kernel mode.
- Before retrying, install the frame's PFN with present = 1, permission RW-, A = 0, and D = 0.
- Restart the instruction. The successful store then normally sets A = 1 and D = 1.

(2) Load from 0x00500100

- This is a major, file-backed page fault because the valid page is nonresident and its file contents are not cached.
- Storage I/O is required, so the thread blocks while the read completes. Frame allocation itself is immediate under the assumptions.

- Install the frame's PFN with present = 1, permission R--, and D = 0, then restart the load. After the successful load, A = 1 and D = 0.

(3) Load from 0x00600020

- This is a major, swap-backed page fault.
- Storage I/O is required to read slot 17, so the thread blocks during the read.
- Install the new PFN with present = 1, permission RW-, and D = 0, then restart the load. Afterward, A = 1 and D = 0.
- Clearing the swap-slot metadata is policy-dependent. The kernel may free slot 17 after page-in, or retain it as an up-to-date backing copy while the resident page remains clean.

(4) Store to 0x00700008

- This is a write-protection fault: the page is present but permits only reading and instruction fetches.
- No storage I/O is required, and the thread does not block for paging. The kernel normally delivers SIGSEGV; its default action terminates the process, and the PTE remains unchanged.
- Under normal unhandled behavior the instruction cannot complete. A custom signal handler could potentially change the mapping and make a later retry possible, so saying it can never be restarted is too absolute.

(5) Load from 0x00800000

- Hardware raises a page fault, after which the kernel classifies the address as unmapped and invalid.
- No storage I/O or paging-related blocking is required. The PTE remains unchanged, and the process normally receives SIGSEGV rather than successfully restarting the instruction.

A non-present PTE may represent a valid demand-zero page, a valid file-backed page, a valid swapped page, or an entirely unmapped address. The kernel must consult VMA or supplemental metadata to determine whether the access is legal and how the page could be materialized.

Question 2 - Eviction and backing-store reasoning

QUESTION

The kernel must reclaim one frame immediately while preserving all required data:

Frame	Current state
A	Clean executable file page
B	Anonymous demand-zero page that has never been written
C	Dirty anonymous page with no swap copy
D	Clean anonymous page whose current contents already exist in swap slot 21
E	Dirty `MAP_SHARED` file page
F	Page pinned for an active DMA operation
G	Dirty `MAP_PRIVATE` file page

Answer:

1. For each frame, decide whether it can be discarded, must be written to the file, must be written to swap, or cannot currently be evicted.
2. Why is “clean versus dirty” alone insufficient to make every decision?
3. Give the correctly ordered eviction procedure for frame C on a multicore system.
4. Suppose C is mapped by two processes. What additional mappings must be invalidated?
5. Why must the kernel complete shootdowns before frame reuse?
6. Why must the frame remain unavailable for reuse until swap output completes?
7. If two threads simultaneously fault on C while it is later being read back, what invariant prevents duplicate frame allocation and duplicate reads?

ANSWER

(1) A: can be evicted without further operations B: can be evicted without further operations C: can be evicted only after being written to swap D: can be evicted without further operations E: can be evicted only after being written to file F: can not be evicted currently G: can be evicted only after being written to swap

(2) The dirty bit indicates that the page has been written since the bit was last cleared. It does not identify the page's backing type or establish whether an authoritative backing location already contains the current contents, so clean/dirty state alone is insufficient.

(3)

- (1) Synchronize access to the page and all its mappings.
- (2) Allocate a swap slot and mark the page as eviction-in-progress.
- (3) Replace every relevant PTE with non-present/in-flight state so that a new page-table walk cannot recreate the old translation.
- (4) Perform the memory-ordering operation required by the architecture.
- (5) Invalidate the corresponding local and remote TLB entries for every virtual alias, and wait for shootdown acknowledgements.
- (6) Start the swap write while keeping the frame pinned or busy.
- (7) Wait for I/O completion, record the completed swap backing, and wake any faulting waiters.
- (8) Only then release the physical frame for reuse.

Performing the shootdown before making the PTEs non-present would be unsafe: another core could walk a still-present PTE and refill the old translation immediately after invalidating it. TLB invalidation normally targets virtual mappings such as `(ASID, VPN)`; reverse-mapping metadata helps find every alias of the physical frame.

(4) must invalidate the PTE in process 1 and the PTE in process 2 for that virtual page; must perform TLB shootdown to all CPU cores that have cached TLB translations for either process's virtual-to-PFN mapping for this page

(5) because otherwise, a CPU could use the stale TLB entry and access the new frame contents via the old virtual address

(6) The storage device may still be reading from the physical frame. If the frame is reused or modified before output completes, the swap slot could receive another page's contents or a mixture of old and new data. The frame must remain busy and unavailable for reuse until completion.

(7) A page lock is part of the mechanism, but the invariant is that at most one page-in operation materializes a particular virtual or backing page at a time. The kernel normally combines a lock with an `in-flight` flag and a wait queue. Other faulting threads wait and recheck the page state after waking; an ordinary spinlock must not be held across blocking storage I/O.

Question 3 - Diagnose a user-buffer/DMA implementation

QUESTION

Consider this flawed system call:

```
ssize_t device_read(void *user_buf, size_t n) {
    spin_lock(&device_lock);

    if (!user_range_is_valid(user_buf, n)) {
        spin_unlock(&device_lock);
        return -EFAULT;
    }

    start_dma(device, user_buf, n);
    sleep_until_dma_finishes();

    spin_unlock(&device_lock);
    return n;
}
```

Assume another thread in the same process may modify its address space concurrently.

1. Identify every important correctness, safety, or liveness problem in this implementation.

2. Why does checking user_range_is_valid once not guarantee that later accesses are safe?
3. Why is passing a user virtual address directly to DMA generally invalid?
4. Why is sleeping while holding device_lock especially problematic if it is a spinlock?
5. Design a corrected implementation based on pinning user pages. Include:
   - when faults are allowed;
   - when pages become pinned;
   - how DMA receives physical mappings;
   - what synchronization protects the device;
   - when the calling thread may sleep;
   - cleanup and unpinning.
6. Give an alternative design using a kernel bounce buffer.
7. Compare the performance and resource trade-offs of pinning versus copying.
8. Construct one deadlock involving a kernel lock and a page fault, then state the invariant that prevents it.

ANSWER

(1) The implementation has several independent problems:

- user_range_is_valid() creates a check-then-act race. Another thread can unmap or reprotect the range after validation.
- The range may span several pages, some of which may be nonresident or not writable. device_read writes into the user buffer, so write permission is required across the entire range.
- Calculating the end of the range may overflow if user_buf + n is not checked safely.
- A DMA device generally cannot use an arbitrary process virtual address. It needs a DMA mapping, normally expressed through bus addresses, an IOMMU address, or a scatter-gather list.
- The pages are not pinned, so their frames may be evicted or reassigned while the device is using them.
- The thread sleeps while holding a spinlock. If the completion path needs the same lock, the operation deadlocks; even without that dependency, sleeping while holding an ordinary spinlock is invalid.
- Request lifetime, partial completion, cancellation, failure cleanup, and DMA mapping cleanup are not handled.

(2) This is a check-then-act race. Validation establishes only that the range was valid at the instant it was checked; it does not stabilize the mappings, permissions, residency, or physical frames throughout the DMA operation.

(3) A raw process virtual address is generally not a valid DMA address. Without an IOMMU, the device normally receives physical or bus addresses, often as a scatter-gather list. With an IOMMU, it may receive an I/O virtual address translated by the IOMMU. In neither case should the kernel pass an arbitrary user virtual address directly to the device.

(4) An ordinary spinlock protects only a short, non-sleeping critical section and may disable preemption or interrupts. Sleeping while holding it is invalid, not merely inefficient. A direct deadlock occurs if the DMA completion handler needs `device_lock`: the caller waits for completion while retaining the lock, while the handler cannot acquire the lock needed to report completion.

(5)

```
ssize_t device_read(void *user_buf, size_t n) {
    struct page_list pages = {0};
    struct dma_mapping mapping = {0};
    struct request req;
    int ret;

    if (!valid_nonoverflowing_range(user_buf, n))
        return -EFAULT;

    /* May fault and sleep, so no device spinlock is held here. */
    ret = pin_user_pages(user_buf, n, WRITE_ACCESS, &pages);
    if (ret < 0)
        return -EFAULT;

    ret = dma_map_pages(device, &pages, user_buf, n,
                        DMA_FROM_DEVICE, &mapping);
    if (ret < 0)
        goto unpin;
```

```
    init_completion(&req.completion);

    spin_lock(&device_lock);
    ret = submit_dma_request(device, &mapping, n, &req);
    spin_unlock(&device_lock);

    if (ret < 0)
        goto unmap;

    /* No spinlock is held while the caller sleeps. */
    wait_for_completion(&req.completion);
    ret = req.result;

    if (ret > 0)
        mark_written_user_pages_dirty(&pages, ret);

unmap:
    dma_unmap_pages(device, &mapping);
unpin:
    unpin_user_pages(&pages);
    return ret;
}
```

Pinning must occur with write intent before acquiring the device spinlock. The DMA mapping must describe page offsets and partial pages, and the pages must remain pinned until completion. Every success and failure path must unmap the DMA mapping, unpin all pages that were pinned, and release references. The request's completion object must be initialized before submission to avoid losing a fast completion event.

(6)

```
ssize_t device_read(void *user_buf, size_t n) {
    struct dma_buffer bounce;
    struct request req;
    int ret;

    ret = dma_alloc_buffer(device, n, &bounce);
    if (ret < 0)
        return ret;

    init_completion(&req.completion);

    spin_lock(&device_lock);
    ret = submit_dma_request(device, bounce.dma_address,
                             n, &req);
    spin_unlock(&device_lock);

    if (ret < 0)
        goto free_buffer;

    wait_for_completion(&req.completion);
    ret = req.result;

    /* Device read: device -> bounce buffer -> user buffer. */
    if (ret > 0 && copy_to_user(user_buf, bounce.cpu_address, ret))
        ret = -EFAULT;

free_buffer:
    dma_free_buffer(device, &bounce);
    return ret;
}
```

For `device_read`, data flows from the device to the kernel bounce buffer and only then to user memory. No preliminary user-to-kernel copy is needed. The bounce buffer must come from a DMA-safe allocation or mapping API; ordinary kernel allocation plus `virt_to_phys` does not necessarily describe a contiguous or device-accessible region. `copy_to_user` may fault and sleep, but DMA has finished and no device spinlock is held.

(7)

Pin user pages	Use a bounce buffer
Avoids an additional data copy and is attractive for large transfers	Adds a copy and consumes memory bandwidth
Pins user frames, reducing the pager's available choices	Consumes kernel DMA-buffer memory instead

Pin user pages	Use a bounce buffer
Requires page references, partial-failure cleanup, and scatter-gather or IOMMU mapping	Keeps the device-facing memory under kernel control and simplifies user-memory lifetime
Can create memory pressure if pages remain pinned too long	Often simpler for small transfers, but large temporary buffers may fail or waste memory

A bounce buffer still must be explicitly DMA-safe; ordinary kernel memory is not automatically contiguous or suitable for every device.

(8) One concrete deadlock is:

```
T1 holds filesystem_lock
T1 calls copy_from_user()
the user page is a nonresident file-backed page
the page-fault handler needs filesystem_lock to read that page
T1 cannot finish copy_from_user() until the fault completes
the fault cannot complete until T1 releases filesystem_lock
```

The appropriate invariant is: do not allow a potentially blocking page fault while holding a non-sleepable lock or any resource required by the page-fault path. It is not universally forbidden to hold every kind of lock during a faultable operation; the relevant questions are whether the thread may sleep and whether the fault handler depends on a resource the thread currently owns.

Lesson 14 - Replacement and locality

Question 1 - Replacement trace and Belady's anomaly

QUESTION

Use the reference string:

```
1 2 3 4 1 2 5 1 2 3 4 5
```

Assume all frame sets begin empty.

1. Trace FIFO with three frames. Give the frame contents after every reference, identify every victim, and count faults.
2. Trace FIFO with four frames and count faults.
3. Identify and explain the anomaly.
4. Trace LRU with three frames and count faults.
5. Trace OPT with three frames and count faults.
6. At the first reference to page 4, explain why FIFO, LRU, and OPT may select different victims.
7. Which of FIFO, LRU, and OPT have the stack property? Use that property to explain why some can never exhibit Belady's anomaly.
8. Suppose page 5 is dirty and every other page is clean. Do the basic textbook forms of these policies change their decisions? Explain why a practical kernel might.

You may use a compact table rather than full sentences for the traces.

ANSWER

(1) FIFO with 3 frames:

```
Frame: 1                 Fault
Frame: 1 2               Fault
Frame: 1 2 3             Fault
Frame: 2 3 4 (Victim: 1) Fault
Frame: 3 4 1 (Victim: 2) Fault
Frame: 4 1 2 (Victim: 3) Fault
Frame: 1 2 5 (Victim: 4) Fault
Frame: 1 2 5             Hit
Frame: 1 2 5             Hit
Frame: 2 5 3 (Victim: 1) Fault
Frame: 5 3 4 (Victim: 2) Fault
Frame: 5 3 4             Hit
```

9 faults in total

(2) FIFO with 4 frames:

```
Frame: 1                   Fault
Frame: 1 2                 Fault
Frame: 1 2 3               Fault
Frame: 1 2 3 4             Fault
Frame: 1 2 3 4             Hit
Frame: 1 2 3 4             Hit
Frame: 2 3 4 5 (Victim: 1) Fault
Frame: 3 4 5 1 (Victim: 2) Fault
Frame: 4 5 1 2 (Victim: 3) Fault
Frame: 5 1 2 3 (Victim: 4) Fault
Frame: 1 2 3 4 (Victim: 5) Fault
Frame: 2 3 4 5 (Victim: 1) Fault
```

10 faults in total

(3) FIFO is not a stack algorithm. This trace demonstrates Belady's anomaly: increasing the allocation from three to four frames raises the fault count from 9 to 10 because FIFO's resident sets do not preserve the subset relationship.

(4) LRU with three frames

```
Frame: 1                 Fault
Frame: 1 2               Fault
Frame: 1 2 3             Fault
Frame: 2 3 4 (Victim: 1) Fault
Frame: 3 4 1 (Victim: 2) Fault
Frame: 4 1 2 (Victim: 3) Fault
Frame: 1 2 5 (Victim: 4) Fault
Frame: 2 5 1             Hit
Frame: 5 1 2             Hit
Frame: 1 2 3 (Victim: 5) Fault
Frame: 2 3 4 (Victim: 1) Fault
Frame: 3 4 5 (Victim: 2) Fault
```

10 faults in total

(5) OPT with three frames

```
Frame: 1                 Fault
Frame: 1 2               Fault
Frame: 1 2 3             Fault
Frame: 1 2 4 (Victim: 3) Fault
Frame: 1 2 4             Hit
Frame: 1 2 4             Hit
Frame: 1 2 5 (Victim: 4) Fault
Frame: 1 2 5             Hit
Frame: 1 2 5             Hit
Frame: 3 2 5 (Victim: 1) Fault
Frame: 3 4 5 (Victim: 2) Fault
Frame: 3 4 5             Hit
```

7 faults in total

(6) At the first reference to page 4, FIFO evicts page 1 because it entered first, and LRU also evicts page 1 because it is least recently used. OPT instead evicts page 3: pages 1 and 2 will be used very soon, whereas page 3's next use is much farther in the future.

(7) LRU and OPT have the stack property. After every reference prefix, the resident set produced with `n` frames is a subset of the resident set produced with `n + 1` frames. Therefore, adding frames can never increase their fault counts, although the count may remain equal rather than becoming strictly smaller. FIFO lacks this inclusion property and can exhibit Belady's anomaly.

(8) No. The basic textbook forms treat all evictions as having equal cost, so the dirty bit does not alter their decisions. A practical kernel may prefer a somewhat more recently used clean page over dirty page 5 to avoid or postpone expensive writeback.

Question 2 - Clock, dirty pages, and TLB interaction

QUESTION

A four-frame Clock ring initially contains:

Slot	Page	R	D
0	A	1	0
1	B	0	1
2	C	1	0
3	D	0	0

The Clock hand initially points to slot 0. On insertion, set the new page's reference bit to `1` and advance the hand to the following slot.

Process this access sequence using basic Clock:

```
E, C, F, A, G
```

1. For every access, state whether it hits or faults.
2. On every fault, show each frame inspected by the hand, including reference bits that are cleared.
3. Identify the victim sequence.
4. Give the final pages, reference bits, dirty bits, and hand position.
5. The first basic-Clock victim is dirty even though an unreferenced clean page exists. Explain how an enhanced Clock policy could make a different decision.
6. Why must enhanced Clock eventually permit dirty pages to be selected?
7. How could background writeback improve later replacement decisions?
8. Explain why clearing a PTE's reference bit may require coordination with a cached TLB entry.
9. If page D were pinned, how would the replacement scan have to treat it?

ANSWER

Parts (1)-(4).

The following trace uses an enhanced Clock policy that prefers unreferenced clean pages, clears reference bits while scanning, and postpones unreferenced dirty pages:

```
E FAULTS

HAND: !
SLOT: 0 1 2 3
PAGE: A B C D
Ref : 1 0 1 0
Dirt: 0 1 0 0

HAND:   !
SLOT: 0 1 2 3
PAGE: A B C D
Ref : 0 0 1 0
Dirt: 0 1 0 0

HAND:     !
SLOT: 0 1 2 3
PAGE: A B C D
Ref : 0 0 1 0
Dirt: 0 1 0 0

HAND:       !
SLOT: 0 1 2 3
PAGE: A B C D
Ref : 0 0 0 0
Dirt: 0 1 0 0

E INSERTED, D EVICTED

HAND: !
```

```
SLOT: 0 1 2 3
PAGE: A B C E
Ref : 0 0 0 1
Dirt: 0 1 0 0

C HITS

HAND: !
SLOT: 0 1 2 3
PAGE: A B C E
Ref : 0 0 1 1
Dirt: 0 1 0 0

F FAULTS, F INSERTED, A EVICTED, A FAULTS

HAND:   !
SLOT: 0 1 2 3
PAGE: F B C E
Ref : 1 0 1 1
Dirt: 0 1 0 0

HAND:     !
SLOT: 0 1 2 3
PAGE: F B C E
Ref : 1 0 1 1
Dirt: 0 1 0 0

HAND:       !
SLOT: 0 1 2 3
PAGE: F B C E
Ref : 1 0 0 1
Dirt: 0 1 0 0

HAND: !
SLOT: 0 1 2 3
PAGE: F B C E
Ref : 1 0 0 0
Dirt: 0 1 0 0

HAND:   !
SLOT: 0 1 2 3
PAGE: F B C E
Ref : 0 0 0 0
Dirt: 0 1 0 0

HAND:     !
SLOT: 0 1 2 3
PAGE: F B C E
Ref : 0 0 0 0
Dirt: 0 1 0 0

A INSERTED, C EVICTED, G FAULTS

HAND:       !
SLOT: 0 1 2 3
PAGE: F B A E
Ref : 0 0 1 0
Dirt: 0 1 0 0

G INSERTED, E EVICTED

HAND: !
SLOT: 0 1 2 3
PAGE: F B A G
Ref : 0 0 1 1
Dirt: 0 1 0 0

E: faults (D evicted)
C: hits
F: faults (A evicted)
A: faults (C evicted)
G: faults (E evicted)
```

Thus, the enhanced-Clock victim sequence is:

```
D, A, C, E
```

The final enhanced-Clock state is:

Slot	Page	R	D
0	F	0	0
1	B	0	1
2	A	1	0
3	G	1	0

The hand points to slot 0.

For comparison, the compact basic-Clock trace is:

```
E: fault; clear A.R; evict B; insert E; hand -> C
C: hit; set C.R = 1
F: fault; clear C.R; evict D; insert F; hand -> A
A: hit; set A.R = 1
G: fault; clear A.R; clear E.R; evict C; insert G; hand -> F
```

Basic-Clock victims are `B, D, C`. Its final state is:

Slot	Page	R	D
0	A	0	0
1	E	0	0
2	G	1	0
3	F	1	0

The hand points to slot 3. The key difference is the first victim: basic Clock evicts unreferenced but dirty B, whereas enhanced Clock continues scanning and finds unreferenced clean D.

(5) An enhanced Clock policy prefers unreferenced clean pages. As long as an eligible `(R=0, D=0)` page exists, it can avoid the immediate writeback cost of evicting an unreferenced dirty page.

(6) Dirty pages must eventually remain eligible; otherwise, a workload in which all candidate pages are dirty could leave the pager unable to select any victim. Even before that extreme, refusing all dirty victims could force excessive scanning.

(7) Background writeback can turn infrequently modified dirty pages into clean pages before memory becomes urgent, creating inexpensive future victims and reducing fault-path latency.

(8) After the kernel clears the PTE's reference bit, a later access may hit in the TLB and avoid another page-table walk. Depending on the architecture, the cleared PTE bit may therefore remain zero even though the page was accessed. The OS must use the architecture's defined reference-bit synchronization or TLB-invalidation procedure.

(9) The scan must skip a pinned page even if its reference bit is zero. Reusing it could corrupt an active DMA or other operation that depends on the frame remaining stable. If every eligible frame is pinned, reclamation must wait, search another memory domain, or fail rather than reuse one of those frames.

Question 3 - Working sets and thrashing

QUESTION

A machine has 12 allocatable user frames and three active processes:

Process	Current working-set estimate	Observed page-fault rate
P1	5 frames	25%
P2	4 frames	1%
P3	6 frames	30%

Assume a desirable fault-rate range of 2%-10%.

1. Can all three estimated working sets fit simultaneously? What does the result suggest?

2. Under local replacement with four frames per process, predict the behavior of each process.
3. Under unrestricted global replacement, explain how P1 and P3 might interfere with P2 even though P2 initially has enough frames.
4. Apply a PFF-style policy. Which processes appear under-allocated or potentially over-allocated?
5. Is taking frames from P2 necessarily sufficient or wise? Use the working-set estimates in your argument.
6. Propose one load-control action that can restore a feasible total working set. State which process you would suspend and defend the choice; more than one answer may be reasonable.
7. Suppose CPU utilization is low, the paging device is saturated, and most runnable work is waiting for page-ins. Should the scheduler admit another process? Explain.
8. Compare local and global replacement for this workload in terms of throughput, isolation, and predictability.
9. Would replacing the storage device with a faster SSD eliminate the underlying cause of thrashing? Distinguish reduced fault cost from sufficient memory capacity.

ANSWER

(1) No. The combined working-set estimate is `5 + 4 + 6 = 15` frames, which exceeds the 12 allocatable user frames. Therefore, the system cannot keep all three estimated working sets resident simultaneously. This strongly suggests sustained page-fault pressure and possible thrashing, although page faults are not literally avoidable even when all working sets fit because compulsory faults can still occur.

(2) Under local replacement with four frames per process, P2 should have a low steady-state page-fault rate because its four-frame working set fits its allocation. P1 and P3 are short of one and two frames, respectively, so they will repeatedly page in needed pages and evict other pages from their own allocations.

(3) Under unrestricted global replacement, faults from P1 and P3 may select P2's frames as victims. P2 can then lose the allocation that currently contains its working set, so its page-fault rate may rise even though it was previously stable.

(4) Under page-fault-frequency control, a process with a persistently high fault rate should receive more frames if any are available, while a process with a persistently very low fault rate may be a candidate for cautious reclamation. P1 and P3 appear under-allocated. P2 is only *potentially* over-allocated: its low fault rate is evidence that four frames are sufficient, but its estimated working set is also four frames, so reclaiming one may destabilize it. The OS should use measurements over a meaningful window and can probe conservatively rather than assuming a low current fault rate proves excess capacity.

(5) Immediately taking a frame from P2 is not well justified. Its estimated working set exactly equals its allocation, so all four frames may be necessary to sustain its low fault rate. Reducing it to three frames could sharply increase its faults and merely transfer memory pressure from P1 or P3 to P2.

(6) Suspending P3 is a defensible choice because it has the largest estimated working set and the highest observed fault rate. The remaining active working sets require only `5 + 4 = 9` frames, leaving three frames of slack within the 12-frame capacity. Suspending P1 would also restore feasibility because `4 + 6 = 10`, so a real policy should additionally consider process priority, progress already made, and suspension or swap cost.

(7) The admission controller should not admit another memory-intensive process. The system is already thrashing: processes spend much of their time waiting for paging I/O, so CPU utilization can be low even though the workload is excessive. Admitting more work would increase memory pressure; the controller should instead reduce the degree of multiprogramming.

(8) Local replacement provides isolation and predictable per-process allocations: P1 and P3 cannot evict P2's resident pages. However, it can trap a process in an undersized allocation and can leave frames poorly distributed or temporarily underused. Global replacement adapts allocations dynamically and can improve aggregate throughput when memory demand varies, but under overload one process's faults can steal frames from another, spread interference, and make formerly stable processes thrash as well.

(9) No. A faster SSD lowers the service time of each page fault, but it does not repair the capacity mismatch between the combined active working set and physical memory. The system will still repeatedly evict and reload pages. Thrashing therefore persists, although each fault and the resulting slowdown may be less expensive.

Lesson 15 - Sharing and mapped memory

Question 1 - Copy-on-write `fork()`

QUESTION

Consider process `P` with three writable anonymous pages immediately before `fork()`:

Virtual page	Contents	Physical frame
`x`	10	`F1`
`y`	20	`F2`
`z`	30	`F3`

Assume:

- fork() uses copy-on-write.
- Immediately after fork(), parent P and child C map all three frames through read-only COW PTEs.
- A COW write fault normally allocates a new frame, copies the old contents, remaps only the faulting process's page as writable, and adjusts reference counts.
- If a COW page's frame has reference count 1, the kernel may simply make that existing mapping writable without copying.
- Ignore eviction and assume allocations succeed unless stated otherwise.

Execute these operations in order:

```
1. C reads x
2. C writes y = 21
3. P writes y = 22
4. P forks a second child D
5. D writes x = 11
6. C exits
7. P writes z = 31
```

Answer:

1. Immediately after the first `fork()`, how many physical data frames hold `x`, `y`, and `z`? Give each frame's reference count. Why must both processes' PTEs normally be write-protected?
2. For every operation above, identify whether it causes:
   - an ordinary read;
   - a COW protection fault;
   - a frame allocation and copy;
   - or only a PTE-permission change.

Track the mappings and contents visible to `P`, `C`, and `D` after every operation.

3. Give the final contents observed by each surviving process and the final reference count of every remaining data frame.
4. At operation 2, suppose frame allocation fails. What state must remain unchanged so that the failed COW fault is atomic from the process's point of view?
5. Why would changing the parent's PTEs to read-only during `fork()` be insufficient on a multicore system if another CPU retained stale writable TLB entries?
6. Describe one optimization that can avoid an unnecessary copy after the other sharers have exited. What kernel metadata is required to apply it safely?

ANSWER

(1) Immediately after the first `fork()`, there are still three physical data frames:

```
F1: x = 10, refcount = 2
F2: y = 20, refcount = 2
F3: z = 30, refcount = 2
```

Both processes' PTEs are read-only and marked COW. A write therefore triggers a protection fault so that the kernel can preserve address-space isolation instead of allowing one process to modify a frame still visible to the other.

(2) The execution is:

```
1. C reads x
```

This is an ordinary read. All mappings remain read-only COW:

```
P: x -> F1 (10), y -> F2 (20), z -> F3 (30)
C: x -> F1 (10), y -> F2 (20), z -> F3 (30)

2. C writes y = 21
```

This causes a COW protection fault, allocation of `F4`, and a copy from `F2` to `F4`. Only C's mapping is remapped writable:

```
P.y -> F2 (20), R-/COW, refcount 1
C.y -> F4 (21), RW,     refcount 1
```

P's PTE remains read-only COW even though P is now the sole owner of `F2`.

```
3. P writes y = 22
```

This causes another COW protection fault. Because `F2.refcount == 1`, the kernel does not allocate or copy a frame; it merely changes P's PTE to writable and retries the instruction:

```
P.y -> F2 (22), RW
C.y -> F4 (21), RW

4. P forks a second child D
```

P and D now share P's pages through read-only COW mappings. C's private `F4` remains writable:

```
F1: P.x, C.x, D.x       refcount 3
F2: P.y, D.y            refcount 2
F3: P.z, C.z, D.z       refcount 3
F4: C.y                 refcount 1

5. D writes x = 11
```

This causes a COW protection fault, allocation of `F5`, and a copy of `F1`:

```
D.x -> F5 (11), RW,     refcount 1
P.x -> F1 (10), R-/COW
C.x -> F1 (10), R-/COW
F1.refcount: 3 -> 2

6. C exits
```

C's mappings are removed:

```
F1.refcount: 2 -> 1
F3.refcount: 3 -> 2
F4.refcount: 1 -> 0; F4 is freed
```

P's `x` mapping normally remains read-only COW. The kernel need not search for and update P's PTE merely because C exited.

```
7. P writes z = 31
```

P and D still share `F3`, so this causes a COW fault, allocation of `F6`, and a copy:

```
P.z -> F6 (31), RW,     refcount 1
D.z -> F3 (30), R-/COW, refcount 1
```

D's mapping remains read-only COW. If D later writes it, the handler can observe that `F3.refcount == 1` and restore write permission without copying.

(3) The final process-visible contents are:

```
P: x = 10, y = 22, z = 31
D: x = 11, y = 22, z = 30
```

The remaining frame reference counts are:

```
F1: 1
F2: 2
F3: 1
F5: 1
F6: 1
```

`F4` has been freed. Under lazy permission restoration, the final mappings are:

```
P.x -> F1, R-/COW       D.x -> F5, RW
P.y -> F2, R-/COW       D.y -> F2, R-/COW
P.z -> F6, RW           D.z -> F3, R-/COW
```

(4) If allocation fails during a COW fault, the original PTE, its COW and permission bits, the original frame and contents, and all reference counts must remain unchanged. The user process must not observe a partially completed write, and any temporary allocation must be released.

(5) Changing a PTE in memory does not invalidate writable translations already cached by other CPUs. A stale writable TLB entry could modify the shared frame without causing a COW fault. The kernel must invalidate the local entry and perform a TLB shootdown on any other CPU that may cache the parent address space's translation.

(6) When a COW write fault finds that the frame has exactly one mapping owner, the kernel may remove the COW state and make the existing PTE writable without allocating or copying a frame. This requires a COW marker, reliable mapping or ownership counts, and synchronization plus revalidation around the count and PTE transition. A count of one permits this optimization; a count of zero means that no mapping owner remains and the frame may be freed.

Question 2 - `MAP_SHARED`, `MAP_PRIVATE`, and the page cache

QUESTION

A file initially contains one page whose first four bytes are:

```
ABCD
```

Processes `P` and `Q` open the same file independently. Therefore, their descriptors have separate open-file offsets, both initially zero.

They then create these mappings:

```
P: s = mmap(NULL, PAGESIZE,
            PROT_READ | PROT_WRITE,
            MAP_SHARED, fdP, 0);

Q: p = mmap(NULL, PAGESIZE,
            PROT_READ | PROT_WRITE,
            MAP_PRIVATE, fdQ, 0);
```

Assume:

- the kernel has a coherent unified page cache;
- all operations below occur in the stated order;
- there are no errors or external file modifications.

Execute:

```
1. P reads s[0]
2. Q reads p[0]
3. P stores s[1] = 'X'
4. Q reads p[1]
5. Q stores p[2] = 'Y'
6. P reads s[2]
7. The kernel writes back every dirty shared file-backed page
8. Q calls msync(p, PAGESIZE, MS_SYNC)
9. P calls read(fdP, buf, 4)
```

Answer:

1. Describe the likely first-fault paths for steps 1 and 2. Which physical frame may initially be shared? Which structures record the relationship among the file, cached page, and PTEs?
2. After each step from 3 through 6, give:
   - the four bytes visible through s;
   - the four bytes visible through p;
   - whether Q's private mapping still shares the page-cache frame.
3. After step 7, what four bytes are stored in the file? Which modification is eligible for file writeback, and which is not?
4. Does step 8 copy Q's private `Y` into the file? Explain the role of `msync()` for `MAP_SHARED` and `MAP_PRIVATE`.

5. What does step 9 place in `buf`, and what is `fdP`'s resulting file offset? Why do loads and stores through `mmap()` not change an open-file offset?
6. Compare eviction for:
   - an unmodified shared file-backed page;
   - a dirty MAP_SHARED page;
   - a privately modified COW page.

For each case, state whether writeback is necessary and where the contents can later be recovered.

7. Suppose P executes `munmap(s, PAGESIZE)` before background writeback. Must the dirty shared modification disappear? Explain the distinction between a process's mapping lifetime and the cached page's dirty/writeback lifetime.

ANSWER

(1) At step 1, P finds a non-present PTE and raises a page fault. The kernel validates the access against P's VMA, looks up the page-cache entry keyed by `(file object, page index)`, allocates a frame and reads the file page if it is not already cached, and installs P's `MAP_SHARED` PTE. A writable PTE is acceptable in this simplified system if hardware dirty-bit tracking and writeback synchronization are available.

At step 2, Q also faults on a non-present PTE. The kernel finds the existing page-cache entry and maps the same physical frame into Q's address space through a read-only COW PTE because Q's mapping is `MAP_PRIVATE`.

The relevant structures are:

```
VMA:
    virtual range -> file object, file offset, MAP_SHARED/MAP_PRIVATE

page-cache entry:
    (file object, page index) -> cached physical frame

PTE:
    virtual page -> physical frame + permissions/COW metadata
```

(2) The mapping-visible contents are:

After step	`s` sees	`p` sees	Does Q still share the page-cache frame?
3: P stores `X`	`AXCD`	`AXCD`	Yes
4: Q reads	`AXCD`	`AXCD`	Yes
5: Q stores `Y`	`AXCD`	`AXYD`	No; the store triggers COW
6: P reads	`AXCD`	`AXYD`	No

Before Q's private write, both mappings observe the shared page-cache frame. At step 5, Q receives a private anonymous copy containing `AXYD`.

(3) Step 7 writes `AXCD` back to the file. P's `X` modified the dirty `MAP_SHARED` page-cache page and is eligible for file writeback. Q's `Y` exists only in Q's private COW copy, so it is not written to the file; if evicted, that private state must instead be preserved as anonymous memory, normally in swap.

(4) No. `msync(p, PAGESIZE, MS_SYNC)` does not copy Q's private `Y` into the file. `MS_SYNC` can wait for dirty shared file-backed modifications to be propagated, but private COW modifications remain private. A clean page has no modified contents to flush, and this call does not use `MS_INVALIDATE`.

(5) Step 9 places `AXCD` in `buf` and advances `fdP`'s open-file offset from 0 to 4. Loads and stores through `mmap()` access memory through virtual mappings rather than performing descriptor I/O, so they do not consume or change an open-file offset.

(6) Eviction behavior is:

Page type	Required handling before final discard	Recovery source
Clean file-backed page	No writeback is needed	Read the file again

Page type	Required handling before final discard	Recovery source
Dirty `MAP_SHARED` page	Write modified contents to the file and retain/pin the frame until writeback completes	Read the updated file again
Privately modified COW page	No file writeback, but modified anonymous contents must be preserved, normally by writing them to swap	Swap or another retained anonymous backing copy

(7) No. `munmap()` removes P's PTEs and mapping references; it does not discard the dirty cached modification. The page can remain alive through page-cache ownership, other mappings, dirty/writeback state, or in-flight I/O references. Mapping lifetime is therefore separate from cached-page and writeback lifetime.

Question 3 - Designing a correct file-backed mapping subsystem

QUESTION

A simplified kernel uses a unified page cache indexed by:

```
(file object, page offset)
```

A proposed implementation follows these rules:

```
A. Every mmap page fault allocates a new frame and reads the file page into it.

B. MAP_SHARED writable mappings install writable PTEs immediately.

C. MAP_PRIVATE writable mappings also install writable PTEs pointing
   directly to the page-cache frame.

D. Eviction may free a dirty page-cache frame immediately after submitting
   an asynchronous disk write that still uses that frame as its source buffer.

E. munmap removes the process's PTEs and immediately frees every
   corresponding frame.

F. The writeback path holds a page-cache spinlock while synchronously
   waiting for disk I/O to complete.
```

Answer:

1. For each rule A-F:
   - decide whether it is correct;
   - if incorrect, give a concrete failure or violated invariant;
   - propose a minimal correction.
2. State a compact set of invariants for a correct design covering:
   - uniqueness and lookup of resident page-cache pages;
   - frame and page reference counts;
   - dirty state and writeback ownership;
   - COW isolation for private mappings;
   - frame lifetime during asynchronous I/O.
3. Construct a race involving two simultaneous faults on the same absent shared file page. Show how a naive check-then-load implementation can create duplicate cached pages or lose a modification. Then describe a synchronization strategy that prevents this without holding a spinlock during disk I/O.
4. Construct a race between writeback and a concurrent store to the same `MAP_SHARED` page. Why is clearing the dirty bit at the wrong time dangerous? Give one correct high-level protocol using page locking, versioning, or temporary write protection.
5. Why should a page-cache lock protect metadata only briefly instead of being held while waiting for disk I/O? Distinguish:
   - the safety problem;
   - the performance problem;
   - the possible deadlock problem.

6. A process faults on a file offset that was valid when its mapping was created, but another process has since truncated the file below that offset. What must the fault handler revalidate? Why is checking only the mapping's original length insufficient?

ANSWER

(1) Rule A is incorrect because every fault cannot independently allocate and load a page. That would violate the unified-page-cache invariant and allow duplicate frames for the same `(file object, page index)`. A simple lookup followed later by insertion is still racy. The kernel should atomically look up or install one unique cache entry in a `loading` state, release the cache lock during disk I/O, then publish completion and wake threads waiting on that entry.

Rule B is acceptable under the simplified assumptions. A writable `MAP_SHARED` PTE may point directly to the page-cache frame if hardware dirty tracking and suitable writeback synchronization are available. Some kernels temporarily write-protect such mappings for first-write detection or writeback coordination, but that is an implementation choice.

Rule C is incorrect. A writable `MAP_PRIVATE` PTE must not modify the shared page-cache frame. The kernel should initially install a read-only COW PTE; a write fault allocates a private frame, copies the contents, and remaps only the faulting process's page as writable.

Rule D is incorrect because the asynchronous device may still be reading the frame after the allocator reuses it, causing use-after-free and corrupted writeback. The I/O path must pin the frame or hold an additional I/O reference until completion.

Rule E is incorrect. `munmap()` drops the process's PTEs and mapping references, but the frame may still be owned by the page cache, mapped elsewhere, dirty, pinned, or involved in I/O. Reclamation is safe only after all relevant mapping, cache, pin, and I/O references are gone and any dirty contents have been preserved. A file page may have zero process mappings while deliberately remaining cached.

Rule F is incorrect. Merely replacing the spinlock with a mutex is insufficient because holding a global cache lock across disk I/O would still serialize unrelated operations and could deadlock with a completion path that needs the same lock. The kernel should briefly lock the metadata, mark and pin the page for writeback, release the lock during I/O, then reacquire it to publish completion and update state.

Lesson 16 - I/O architecture

Question 1 - Trace a blocking DMA read

QUESTION

Thread `T` on CPU 0 calls:

```
read(fd, user_buf, 8192);
```

Assume:

- the data is not cached;
- the driver allocates a DMA-safe kernel bounce buffer K;
- the device performs DMA into K;
- T blocks after submission;
- thread S runs on CPU 0 while the device works;
- the completion interrupt arrives on CPU 1;
- the interrupt handler records completion and wakes T;
- when T eventually runs, it copies K into user_buf, frees K, and returns;
- user_buf is initially valid and resident.

Answer:

1. Trace the complete execution from the system call to its return. At each important point, state:
    - which CPU and thread are executing;
    - whether execution is in user or kernel mode;
    - whether T is running, blocked, or ready;

- where the data is located.

2. Identify every mode switch and every definite context switch. Explain why the completion interrupt does not necessarily cause CPU 1 to switch immediately to `T`.
3. State the ownership and lifetime invariant for `K`. At what exact point may it be freed?
4. Why should the interrupt handler not call `copy_to_user()` directly? What additional work would be required if the device instead performed DMA directly into `user_buf`?

ANSWER

(1) The complete execution is:

```
**(1)** T runs in user mode on CPU 0 and calls read().

**(2)** CPU 0 enters kernel mode. T is still running while it validates the request, allocates the
    DMA-safe bounce buffer K, prepares the request, and submits it.

**(3)** T blocks waiting for completion. The scheduler context-switches CPU 0 from T to S.

**(4)** S runs while the device performs DMA from the device into K. T remains blocked.

**(5)** The device completes and raises an interrupt on CPU 1. The handler acknowledges completion,
    records the result, and changes T from blocked to ready. The completed data is now in K.

**(6)** A scheduler later selects T on CPU 0 or CPU 1. T resumes inside read() in kernel mode, copies the
    completed bytes from K to user_buf, performs cleanup, and returns.

**(7)** T resumes in user mode with the data in user_buf.

The prompt does not guarantee which CPU eventually runs T. It also does not fully specify what CPU 1 or S
    was doing before the interrupt.
```

(2) The definite transitions are:

```
- The read() system call changes CPU 0 from user mode to kernel mode without a context switch.
- When T blocks and S is selected, CPU 0 performs a context switch from T to S.
- Interrupt entry on CPU 1 changes user mode to kernel mode only if CPU 1 interrupted user code. If it
    interrupted kernel code, no privilege-mode change is required. Entering an interrupt handler is not
    inherently a thread context switch.
- Waking T performs blocked-to-ready, not ready-to-running.
- When a scheduler later selects T, that CPU performs a context switch from its current thread to T.
- Returning from read() changes T from kernel mode to user mode without a context switch.

The completion interrupt need not switch immediately to T. The scheduler may keep running the interrupted
    thread or choose another ready thread according to its policy.
```

(3) K must remain allocated, DMA-mapped, and unavailable for reuse from request submission until the device has definitively completed or cancelled the request. After completion, it must remain alive until the kernel has finished consuming or copying its contents and no device, handler, worker, or thread retains a reference.

```
K may be unmapped and freed after DMA is complete and the copy attempt has finished. Successful copying
    is not required: if copy_to_user() fails, K may still be released once no component can access it.
```

(4) A hard interrupt handler should not call copy_to_user() because the operation may fault and sleep, the handler may have interrupted an unrelated address space, and copying a large buffer would make interrupt handling too long.

```
Direct DMA into user_buf requires the kernel to validate the complete writable range, resolve any COW
    mappings, fault in and pin every page, create a scatter-gather list or IOMMU mapping, perform any
    required cache/DMA synchronization, keep the frames and request alive until completion, and finally
    unmap and unpin the pages. Pinning prevents reclamation but does not itself prevent concurrent CPU
    modification, so the interface may also need an ownership or synchronization rule.
```

Question 2 - Polling, interrupts, and DMA

QUESTION

A device completes one request every (5,\mu s) while busy. Handling one interrupt costs (2,\mu s) of CPU time.

Answer:

1. With one interrupt per completion, what fraction of one CPU is spent on interrupt handling? If the device instead interrupts once per batch of ten completions, what is the approximate fraction? Ignore all other costs.

2. Choose a suitable mechanism for each workload and justify it:
   - a keyboard producing a few events per second;
   - a high-speed network queue that remains continuously busy;
   - an SSD that is usually idle but occasionally receives a large request.

Consider interrupts, polling, hybrid polling, and DMA where appropriate.

3. Explain why these two statements are not contradictory:
   - “The device uses DMA.”
   - “The driver polls for completion.”
4. Interrupt coalescing reduces CPU overhead but can increase latency. Explain this trade-off and identify one workload that should favor small batches and one that should favor large batches.

ANSWER

(1) With one interrupt every 5 microseconds and a 2-microsecond interrupt cost:

```
   2 / 5 = 40%

With one interrupt for every ten completions:

   2 / (10 * 5) = 4%

Thus interrupt batching reduces the modeled interrupt CPU cost from 40% to 4% of one CPU.
```

(2) Suitable choices are:

```
- Keyboard: interrupts. Events are rare, so continuous polling would waste CPU time.
- Continuously busy high-speed network queue: polling or an interrupt-to-polling hybrid. An interrupt can
    announce new traffic; the driver can then suppress further interrupts, poll a bounded batch, and
    re-enable interrupts when the queue becomes quiet.
- Usually idle SSD with occasional large requests: DMA plus interrupt completion. DMA avoids CPU copying
    for the large transfer, while interrupts avoid wasting a CPU during idle periods. Hybrid polling may
    help an unusually latency-sensitive workload, but request size alone does not justify polling.
```

(3) DMA and polling describe different dimensions of I/O. DMA determines how data moves between the device and memory. Polling determines how the CPU discovers completion. A device may therefore transfer data with DMA while the driver repeatedly checks a completion register or queue.

(4) Interrupt coalescing waits for a count threshold or timer so that one interrupt reports several completions. Larger batches reduce per-completion interrupt overhead and improve batching, but early completions wait longer before notification.

```
Latency-sensitive interactive networking or audio should favor small batches. Bulk storage or
    high-throughput network processing can favor larger batches.
```

Question 3 - Review a broken driver

QUESTION

Consider this simplified driver:

```
ssize_t device_read(void *user_buf, size_t n)
{
    struct request req = { .done = false };

    spin_lock(&device_lock);

    device_dma_start(virt_to_phys(user_buf), n, &req);

    while (!req.done)
        cpu_relax();

    spin_unlock(&device_lock);
    return n;
}

void device_interrupt(void)
{
    spin_lock(&device_lock);
```

```
    current_request->done = true;
    spin_unlock(&device_lock);
}
```

Answer:

1. Identify at least four independent correctness or design problems. For each, explain a concrete failure rather than merely naming the issue.
2. Give a safe high-level redesign using either:
   - pinned user pages with scatter-gather/IOMMU mapping; or
   - a DMA-safe kernel bounce buffer.

You do not need to write complete C code.

3. State the cleanup order after successful completion and after submission failure. Include request lifetime, DMA mappings, pinned pages or bounce buffers, and wakeup/completion state.
4. Construct the deadlock or non-progress execution caused by `device_lock`. Explain how releasing the lock before waiting and using a completion object resolves it.

ANSWER

(1) Independent problems include:

```
- user_buf is neither validated, faulted in, COW-resolved, nor pinned, so its backing frames may be
    absent, replaced, or otherwise unsuitable while DMA is active;
- virt_to_phys(user_buf) is not a valid general translation for a user virtual address, and a multi-page
    user range may occupy noncontiguous physical frames;
- device_read() retains device_lock while waiting for req.done, but the interrupt handler needs the same
    lock before it can set req.done, producing deadlock or permanent non-progress;
- the busy-wait consumes a CPU instead of blocking the thread;
- request completion lacks a proper atomic or completion protocol and its required memory ordering;
- submission errors, device errors, short transfers, interruption, timeout, and cancellation are not
    handled; the function always returns n;
- a stack request becomes unsafe if an error or cancellation path returns before the device and interrupt
    handler have released all references;
- a single global current_request does not naturally support multiple concurrent requests.
```

(2) A safe bounce-buffer design is:

```
   allocate a request object with a stable lifetime
   allocate/map a DMA-safe kernel buffer and obtain its device address
   initialize a completion object

   briefly lock queue metadata
   submit the request using the DMA address
   publish the request if submission succeeds
   unlock queue metadata

   if submission failed before device ownership:
       release the DMA mapping and buffer
       release the request
       return the error

   wait on the completion object without holding device_lock

   read the recorded status and transferred-byte count
   synchronize the DMA buffer for CPU access if required
   copy the completed bytes to user_buf in process context
   release the DMA mapping and buffer
   release the final request reference
   return the result

The interrupt handler acknowledges the device, identifies the completed request, records its status and
    byte count, publishes completion with the necessary memory ordering, and signals the completion
    object. It does not access arbitrary user memory.

An ordinary kernel allocation is not automatically a DMA-safe buffer, and the device normally requires a
    DMA/bus address rather than a kernel virtual address. Under Linux-style semantics, copy_to_user()
    returning zero means success; a nonzero return means that some bytes were not copied.
```

(3) After successful completion:

```
**(1)** The device stops accessing the buffer.
```

```
**(2)** The handler acknowledges completion and records status and the transferred-byte count.

**(3)** The driver performs any required DMA completion or CPU-synchronization operation.

**(4)** Completion is published and the waiter is awakened.

**(5)** The waiting thread copies the completed bytes to user space.

**(6)** Whether that copy succeeds or fails, the DMA mapping is released and the bounce buffer is freed.

**(7)** The request is destroyed only after the device, queue, handler, and waiting thread have released
    all references.

If submission failure guarantees that the device never accepted the request, the caller removes any
    unpublished queue metadata, releases the DMA mapping and buffer, releases the request, and returns
    the error. If acceptance is ambiguous or the operation may already be in flight, the driver must
    first cancel or quiesce the request and confirm that the device can no longer access its memory.
```

(4) The non-progress execution is:

```
   reader acquires device_lock
   reader submits the request
   reader waits for req.done while retaining device_lock
   device completes and raises an interrupt
   handler tries to acquire device_lock and cannot
   handler cannot set req.done
   reader cannot stop waiting until req.done becomes true

Releasing the lock before waiting is necessary but not sufficient: a bare flag and sleep can still suffer
    a data race or lost wakeup. A completion object supplies a remembered completion event and the
    required synchronization. The reader publishes the request under the lock, releases the lock, and
    waits. The handler briefly updates the result, releases any metadata lock, and signals completion. If
    the signal occurs before the wait begins, the later wait returns immediately.
```

Lesson 17 - Storage systems

Question 1 - HDD scheduling

QUESTION

An HDD has cylinders numbered 0-99. The head starts at cylinder 50. The pending queue, in arrival order, is:

```
90, 10, 55, 58, 44
```

For LOOK, assume the current direction is upward.

1. Give the service order and total head movement for:
   - FCFS;
   - SSTF;
   - LOOK.

Break ties, if any, by earlier arrival.

2. Which policy performs best for this particular queue? Explain why this single example does not prove that it is always best.
3. Construct a short arrival pattern that can starve a request at cylinder 90 under SSTF while the head remains near low-numbered cylinders. Explain why LOOK or SCAN provides a stronger waiting-time property.

ANSWER

```
FCFS:
    head movement:
    50 -> 90 -> 10 -> 55 -> 58 -> 44
    total movement = 182

SSTF:
    head movement:
    50 -> 55 -> 58 -> 44 -> 10 -> 90
    total movvement = 136

LOOK:
    head movement:
    50 -> 55 -> 58 -> 90 -> 44 -> 10
    total movement = 120
```

(2) LOOK performs best for this particular pattern because the total movement of head is the lowest among the three. This single example does not prove that it is always the best because different arriving order will affect the result, and in reality, the pending queue is changing from time to time. it only shows the result for the particular workload. for example, for a queue `49 97 98 99`, LOOK will move in order `50 -> 97 -> 98 -> 99 -> 49`, while `50 -> 49 -> 97 -> 98 -> 99` has a shorter total movement.

(3) my short arrival pattern:

```
the initial pending queue:
51 52 53 54 55 90

the following requests keep coming:
51 52 53 54 55 51 52 53 54 55 ... (constantly asking for cylinder 51~55)
```

it's always "shorter" to seek cylinder 51~55 than cylinder 90, so it will starve. LOOK or SCAN has a stronger waiting-time property for this pattern because the head continues in its chosen direction until it reaches the last pending request or the disk boundary before reversing. The request at cylinder 90 will therefore be served on the upward sweep rather than repeatedly losing to newly arriving nearby requests. This avoids the particular SSTF starvation pattern above; a general waiting-time guarantee still depends on assumptions about arrivals and fairness among requests at the same location.

Question 2 - SSD garbage collection

QUESTION

An SSD garbage-collection victim block contains eight physical pages:

```
3 valid pages
5 invalid pages
0 free pages
```

Assume the controller has enough temporary destination space elsewhere. It copies the three valid pages, erases the victim block, and then accepts four new host-page writes.

1. Counting page programs but not the erase operation, calculate the write amplification for this episode.
2. Explain why repeated small random overwrites generally produce more garbage-collection work than large sequential writes.
3. Explain how both TRIM and overprovisioning can reduce write amplification, and distinguish their mechanisms from wear leveling.
4. A workload receives higher throughput but much worse tail latency when queue depth increases. Explain how both results can occur simultaneously.

ANSWER

(1) write amplification: 7 / 4 = 1.75

(2) An SSD normally handles an overwrite out of place: it programs a new physical page and marks the old page invalid. Small random overwrites tend to scatter invalid pages across many erase blocks. When garbage collection reclaims one of those blocks, it may have to copy many still-valid pages before erasing the block, increasing write amplification and consuming bandwidth. Large sequential writes often produce a more favorable layout: invalid and valid pages are more likely to be grouped, so the controller may find victim blocks with fewer valid pages to copy. Sequential writes can still trigger garbage collection, however; the result also depends on the FTL, utilization, and overprovisioning. They do not

necessarily overwrite every page of a physical erase block.

(3) TRIM: informs SSD that the selected logical ranges may be discarded so that during a garbage collection, the SSD will not have to copy a page which is actually invalid to somewhere else, so the write amplification is reduced. Overprovisioning: provides extra physical space not exposed to the host for relocation and garbage collection. with the extra space, GC can run earlier, pick better victim blocks (low valid count), and avoids being forced to reclaim blocks with lots of valid pages when free space is tight wear leveling: SSDs are not as durable as HHDs. it's designed to bear a certain times of write and erase. if too much writes and erases are concentrated in a certain area of SSD, it will wear out too quickly. In order to increase the endurance of SSDs, the controller has to level wear throughout all blocks instead of putting too much load on certain blocks.

(4) Higher queue depth means more requests are outstanding concurrently. The controller can keep multiple channels and flash dies busy, hiding some device latency and increasing throughput; it need not wait for more requests to arrive. Individual requests may spend longer queued or contend for internal resources, while garbage collection can worsen tail latency. Merging nearby requests is possible in some cases but is not the main explanation for the throughput gain.

Question 3 - RAID design and failure behavior

QUESTION

A system has four identical 2 TiB disks. Assume all disks must participate in the selected array.

1. For RAID 0, RAID 1 using two mirror pairs, RAID 5, and RAID 6, give:
   - usable capacity;
   - the number of arbitrary disk failures the array is guaranteed to tolerate.
2. For a one-block RAID-5 write, describe the read-modify-write sequence. Why can a full-stripe write avoid the two old-data reads?
3. Choose an appropriate layout and justify it for each workload:
   - temporary reproducible data where maximum capacity and throughput matter;
   - important read-heavy data where simple recovery and availability matter more than capacity;
   - a large array that must remain operational after any two disk failures.

Mention at least one limitation of each choice.

ANSWER

(1)

```
RAID 0:
    Capacity: 8 TiB
    can tolerate 0 arbitary disk failures

RAID 1:
    Capacity: 4 TiB
    can tolerate 1 arbitary disk failures
    (may tolerate 2 disk failures if these failues are not in the same mirror pair)

RAID 5:
    Capacity: 6 TiB
    can tolerate 1 arbitary disk failures

RAID 6:
    Capacity: 4 TiB
    can tolerate 2 arbitary disk failures
```

(2)

- (1) read the old data block to be overwritten
- (2) read the old parity block for that stripe
- (3) compute new parity
- (4) write the new data block to its disk
- (5) write the new parity block to parity disk The read-modify-write calculation is new_parity = old_parity XOR old_data XOR new_data, requiring two reads and two writes. A full-stripe write supplies all data blocks of the stripe at once,

so the controller can compute parity directly by XORing the new data blocks. It need not read the old data or old parity.

(3)

- **(1)** temporary reproducible data where maximum capacity and throughput matter:
- RAID 0. It provides maximum usable capacity and high throughput for this workload. It cannot tolerate a disk failure, but the data is reproducible, so that trade-off is acceptable. It does not necessarily deliver the highest throughput for every workload.
- **(2)** important read-heavy data where simple recovery and availability matter more than capacity
- RAID 1. RAID 1 provides easy data recovery and excellent read throughput(can read from either disk in a pair). although it will half the available space, in this case availability is more important than capacity.
- **(3)** large array, survive any two arbitary disk failures
- RAID 6. RAID 6 can tolerate any two disk failures. It uses the capacity equivalent of two disks for redundant information, typically with the parity units distributed across disks rather than two permanently dedicated parity disks. In this four-disk array, that is a 50% capacity cost; RAID availability is not a substitute for backups.

Lesson 18 - File-system interface

Shared assumptions: Assume calls succeed unless failure is explicitly queried. Each requested read returns its full length unless it reaches EOF.

Question 1 - Descriptors versus open file descriptions

QUESTION

Initially `/work/data` contains `ABCDEFGH`, and the process's current directory is `/work`. Calls occur in the stated order:

```
a = open("data", O_RDONLY);
b = dup(a);
c = open("data", O_RDONLY);

read(a, ..., 2);
read(b, ..., 2);
read(c, ..., 3);

fork();  // Both processes retain a, b, and c.
```

The **child runs first**: it calls `read(b, ..., 2)`, closes `a` and `b`, then exits. Only after that, the **parent** calls `read(a, ..., 2)` and `read(c, ..., 2)`.

1. Which descriptors share an open file description? Which refer to the same inode?
2. Give the bytes returned by all six reads, in execution order.
3. Give the final offsets of the parent's open file descriptions. Why did the child's close() calls not invalidate the parent's a or b?

ANSWER

```
a = open("data", O_RDONLY);
b = dup(a);
c = open("data", O_RDONLY);
// a -> file description 1 (points to inode x, offset = 0)
// b -> file description 1 (points to inode x, offset = 0)
// c -> file description 2 (points to inode x, offset = 0)

read(a, ..., 2);
// file description 1 points to inode x, offset = 2
read(b, ..., 2);
// file description 1 points to inode x, offset = 4
read(c, ..., 3);
// file description 2 points to inode x, offset = 3

fork()
// c_a -> file description 1 (points to inode x, offset = 4)
// c_b -> file description 1 (points to inode x, offset = 4)
// c_c -> file description 2 (points to inode x, offset = 3)
// p_a -> file description 1 (points to inode x, offset = 4)
// p_b -> file description 1 (points to inode x, offset = 4)
// p_c -> file description 2 (points to inode x, offset = 3)
```

Child:

```
read(b, ..., 2);
// file description 1 points to inode x, offset = 6

close(a); close(b);
// c_a closed
// c_b closed
// c_c -> file description 2 (points to inode x, offset = 3)
// p_a -> file description 1 (points to inode x, offset = 6)
// p_b -> file description 1 (points to inode x, offset = 6)
// p_c -> file description 2 (points to inode x, offset = 3)

// Child exit also closes c_c. The parent's descriptors remain valid.
```

Parent:

```
read(a, ..., 2);
read(c, ..., 2);
// c_a closed
// c_b closed
// c_c closed when the child exited
// p_a -> file description 1 (points to inode x, offset = 8)
// p_b -> file description 1 (points to inode x, offset = 8)
// p_c -> file description 2 (points to inode x, offset = 5)
```

(1) c_a, c_b (before they're closed), p_a, p_b share an open file description c_c, p_c share an open file description all of them refer to the same inode

(2) bytes read (aren't there six reads?) AB CD ABC EF GH DE

(3) file description 1 (points to inode x, offset = 8) file description 2 (points to inode x, offset = 5) The kernel tracks references to each open file description separately from the inode's hard-link count. Closing a descriptor removes that descriptor's reference; the open file description remains while other descriptors still refer to it. In this exercise's descriptor-level model, after `fork()` there are four references to file description 1. The child's `close(a)` and `close(b)` reduce that count to two, leaving the parent's `a` and `b` valid.

Question 2 - Names and lifetime

QUESTION

Initially `/p/a` is the only hard link to inode **I**, containing `OLD`; there are no open descriptors. Run:

```
link("/p/a", "/p/h");
symlink("a", "/p/s");
fd = open("/p/a", O_RDONLY);
unlink("/p/a");
```

For any inspection below, close temporary descriptors immediately; keep `fd` open.

1. At this point, what happens when you try to open /p/h and /p/s, read through fd, and call readlink("/p/s")? Explain each result.
2. Now create a new regular file /p/a (inode J) containing NEW. Which inode does opening /p/s reach? Which inode does opening /p/h reach? Track I's hard-link count from the start through these steps.
3. Finally, call unlink("/p/h") and close(fd). At which step can I's storage be reclaimed, and why?

ANSWER

```
// "/p/a" hard linked to inode I, containing OLD
// link count of I = 1

link("/p/a", "/p/h");
// "/p/h" hard linked to inode I, containing OLD
// link count of I = 2

symlink("a", "/p/s");
// "/p/s" symbol linked to "a"
// link count of I = 2

fd = open("/p/a", O_RDONLY);
// fd -> file description 1 ->  inode I (containing OLD)
// link count of I = 2

unlink("/p/a");
// "/p/a" no longer exists
// link count of I = 1
```

(1)

```
fd2 = open("/p/h", O_RDONLY);
// open normally
// fd2 -> file description 2 -> inode I (containing OLD)
// "/p/h" is directly linked to inode I (hard link)
close(fd2); // inspection complete; fd2 no longer holds I open

fd3 = open("/p/s", O_RDONLY);
// fails
// when parsing "/p/s", the result is "/p/a" which does not exist

read(fd, ..., n);
// succeeds, returns OLD
// although "/p/a" has been unlinked, inode I still exists (because it is referenced by fd)

readlink("/p/s");
// succeeds, returns "a"; the symbolic link still exists
```

(2) "/p/s" reaches inode J (containing NEW) "/p/h" reaches inode I (containing OLD) (link count of inode I is shown above)

(3)

```
unlink("/p/h");
// link count of I = 0
// but fd -> file description 1 -> inode I
// reference count of I > 0

close(fd);
// now the reference count of I == 0
// I's storage can now be reclaimed because no name is linked to it and no open description references
    it. The temporary fd2 was closed immediately after inspection.
```

Question 3 - A pathname race

QUESTION

A privileged service does this:

```
stat("/jobs/input", &st);
if (allowed(st))
    fd = open("/jobs/input", O_RDONLY);
read(fd, ...);
```

An attacker can change the **final directory entry** `input` between calls, but cannot change `/jobs` or its ancestors.

1. Construct an execution in which the service validates one inode but reads another.
2. Propose a sequence using open, fstat, and-if appropriate-O_NOFOLLOW that ensures the checked object is the object read. State what happens if validation fails.
3. After your successful fstat, the attacker unlinks or replaces /jobs/input. Does that redirect a subsequent read(fd, ...)? Explain using the object model above.

ANSWER

(1)

```
reader process    : stat("/jobs/input", &st);
reader process    : if (allowed(st)) // allowed(st) == true, enter the body
// context switch
malicious process : unlink("/jobs/input");
malicious process : link("/jobs/target", "/jobs/input");
// Assume this link is permitted. /jobs/target is a different inode
// that would fail allowed(), but the service has already checked the old inode.
// context switch
reader process    :     fd = open("/jobs/input", O_RDONLY); // opens the target
reader process    : read(fd, ...);
```

The attacker could instead install a symlink to a forbidden target; the original service would follow it at `open()`.

(2)

```
int fd = open("/jobs/input", O_RDONLY | O_NOFOLLOW);
if (fd < 0) reject();

struct stat st;
if (fstat(fd, &st) < 0 || !allowed(&st)) {
    close(fd);
    reject();
}

read(fd, ...);
close(fd);
```

Here `reject()` terminates this operation. `O_NOFOLLOW` rejects a symlink in the final path component; the stable-ancestor assumption matters.

(3) No, it does not. The fd points to an open file description linked to the original inode. Even if an attacker unlinks or replaces `/jobs/input`, a subsequent `read(fd, ...)` will not be redirected. This preserves the opened object's identity, but does not prevent another process from modifying that same object's contents.

Lesson 19 - File-system implementation

Shared assumptions: Block numbers are zero-based. Parameters are for a toy file system, not Pintos's exact inode layout.

Question 1 - Indexed inode and a sparse write

QUESTION

A file system has 4 KiB blocks and 4-byte block pointers. Each inode has **10 direct pointers, one single-indirect pointer, and one double-indirect pointer**; no triple-indirect pointer. Indirect blocks are the same size as data blocks. Sparse files are supported, and metadata blocks are allocated only when needed.

1. For logical file blocks 9, 10, and 2059, identify the pointer path from the inode. For block 2059, give both double-indirect indices.
2. What is the maximum file size addressable by this inode layout, ignoring the actual disk's capacity?
3. Starting with an empty inode, write one byte at offset 2059 × 4096 + 7. How many physical blocks must be allocated for data and pointer metadata? What must a read of the unwritten gap return? State the resulting logical file size.

ANSWER

(1) inode -> 10th direct pointer -> logical file block 9 inode -> single indirect pointer[0] -> logical file block 10 inode -> double indirect pointer[1] -> inner indirect pointer[1] -> logical file block 2059

(2) number of addressable data blocks = 10 + 1024 + 1024^2 = 1,049,610 highest zero-based logical block number = 1,049,609 maximum file size = 4096 bytes * 1,049,610 = 4,299,202,560 bytes (about 4.004 GiB)

(3) three. The block where the written byte is located Double-indirect first-level metadata block (the outer indirect block) The second-level indirect block pointed to by entry 1 of the first-level block The unwritten gap, including the other unwritten bytes in the newly allocated data block, must read as zeros. The write ends at byte offset `2059 * 4096 + 7`, so the resulting logical file size is `2059 * 4096 + 8 = 8,433,672` bytes.

Question 2 - Find the cache invariants

QUESTION

A buffer cache has two empty slots. Physical block **P** is not cached. A naïve implementation checks “is P cached?”, releases its lock, chooses a slot, reads P from disk, then marks the slot valid.

1. Give an interleaving in which threads T1 and T2 create two cache entries for P. If T1 later modifies one entry, what can go wrong when T2 reads the other? State an invariant and synchronization rule that prevent this.
2. In another execution, T1 is copying bytes from P’s cache entry while T2 evicts that entry and reuses its slot for block Q. What can T1 observe? How should the cache prevent this?
3. A write modifies P in a write-back cache and returns while the disk still has the old value. What should a subsequent read through the cache see? What can you conclude-and not conclude-about recovery after an immediate power failure?

ANSWER

(1)

```
T1: acquire the cache-lookup lock
T1: checks whether P is cached -> negative
T1: releases the lock without reserving P
T2: acquires the cache-lookup lock
T2: checks whether P is cached -> negative
T2: releases the lock without reserving P
T1: choose a slot, read P from disk, mark the slot valid
T2: choose a slot, read P from disk, mark the slot valid
```

if later T1 modified one entry, as T2 holds another copy of the physical block, the cached block will differ in T1 and T2's perspective. the modification of T1 will not be noticed by T2, and we won't know which copy really 'represents' the true block P. The key invariant is that at most one cache entry is associated with physical block P, even while that entry is still `LOADING`. A thread must reserve P in the cache's lookup structure before releasing its lock; another thread finding that reservation waits for the load to finish rather than starting an independent load.

(2) T1 may copy bytes from Q rather than P after T2 reuses the same cache slot. T1 should pin P's cache entry while using it, preventing eviction or reuse until T1 finishes; appropriate entry synchronization must also protect concurrent modifications.

(3) A subsequent read through the coherent cache should see the modified block. After an immediate power failure, the new value is not guaranteed to survive: the disk may still hold the old value if the dirty block was not flushed, while a completed flush could leave the new value. A write-back cache alone does not guarantee durability. Explicit flushing with correct update ordering is one way to address this; backup power is another possible protection. Crash recovery is a separate issue.

Question 3 - Choose the right structures

QUESTION

A toy file system stores each file in **exactly one contiguous extent**, keeps directories as **unsorted arrays of** `(name, inode-number)` **entries**, and tracks free blocks with a bitmap. Its only free data-block locations are `{12, 14, 16, 18}`; every other data-block location is allocated. `/lab/results` currently occupies one block at location 5 and needs to append two blocks.

1. Can this one-extent design extend the file in place or relocate it to a three-block contiguous run? Why? Explain how an indexed inode with two available direct pointers changes the answer.

2. If /lab contains (N) entries, what is the worst-case work to find the name results in the unsorted directory? Suggest a structure that improves large-directory lookup. Would that structure also improve mapping a byte offset within results?
3. Suppose the indexed design successfully appends the two blocks. Name the file-system structures that must change, and one structure that need not change. Does the successful return of a write-back-cache write() prove all those changes are durable? No crash-recovery protocol is required yet.

ANSWER

(1) No, it can't. The file currently occupies one block at location 5. However, the block at location 6 is not available, and there's no three contiguous free blocks, so the file cannot be extended under current circumstances. An indexed inode with two available direct pointers can change the answer by pointing these two pointers to two free datablock locations. This inode organization supports uncontiguous layout of file, which solves the problem of external fragment.

(2) In the worst case, an unsorted array may require checking all N entries to find `results`, so lookup is O(N). A hash index can provide expected O(1) lookup under suitable hashing and load factor, but worst-case collisions can still make it O(N). A sorted array allows O(log N) search but costly updates; a B-tree offers logarithmic lookup and updates with good on-disk locality. it wouldn't improve mapping a byte offset within results. the mapping of a byte offset within a file is determined by the structure of `inode` instead of the layout of a directory file.

(3) the inode for "/lab/results" must change (update block pointers and increase file size) the free block bitmap must change to mark the two newly allocated blocks as allocated the two new data blocks must receive the appended contents the directory file of "/lab" need not change.

The successful return of a write-back-cache `write()` does not prove all those changes are durable. The cached copies may have been modified and marked dirty without yet being written to storage.

Lesson 20 - Crash consistency

Question 1 - What can each write order leave behind?

QUESTION

A file currently has length 4,096 bytes. The file system appends one full block of new bytes **X** using a previously free physical block **B**. Define:

- D: persist X in B;
- M: persist a bitmap update marking B allocated;
- I: persist an inode update pointing logical block 1 to B and setting length to 8,192.

Initially B may contain unrelated old bytes. Assume each of `D`, `M`, and `I` is an atomic block write, that the specified order is enforced by flushes, and that no other operation allocates B during this attempt. There is no journal.

1. With order I → M → D, describe the on-disk state and principal hazard after a crash just after I, and just after I and M.
2. With order D → M → I, describe the state after a crash just after D, and just after D and M. Which structural hazard does this order avoid, and which problem remains?
3. Can ordering alone guarantee that the append is either fully present or fully absent after recovery? Explain what a checker could repair and what it could not reconstruct.

ANSWER

(1) senario 1: a crash just after I; M and D do not take place; on disk state: block B is marked as free because a bitmap update does not take place. however, at the same time, the inode has a block pointer pointing to B and treats B as its logical block 1. principal hazard: users can visit block B via the inode, but the block is marked as free; B maybe allocated to other inodes in the future, and this will cause a pair of hazards: the logical block 1 of the inode may be read / overwritten by other inodes; other inode's logical blocks may be mapped to B and be read / written by the current inode. Moreover, the inode does not know that X isn't persisted in B and believed that the file is intact and is 8192 bytes long.

senario 2: a crash just after I and M, D does not take place; on disk state: block B has been allocated; the inode has a block pointer pointing to B and treats B as its logical block 1. principal hazard: X is not in the new block while the metadata in inode implies that a full block has been appended to the original file. the actual writing of X does not take

place while it seems that the whole operation is complete. (Or in other words, the OS can not infer from inode or other information on disk whether the operation has finished)

(2) senario 1: a crash just after D on disk state: block B is marked as free, although X has been written to it; the inode has not been updated, and B is not part of the file/inode. At this stage, there is no file-system structural inconsistency: the inode cannot reach B, and the bitmap still marks B free. Before a future allocation exposes B's contents to another file, the allocator must initialize or fully overwrite the relevant bytes so that the new file cannot read X; it need not literally erase the whole block.

senario 2: a crash just after D and M on disk state: block B is marked as allocated and stores X; the inode has not been updates, so B is not part of the file/inode. principal hazard: a block leak occurs because B is marked as allocated while no actual inode(file) really holds it. (the issue is not as significant as the two in the question above, but it can still leave less available space in the storage.)

(3) Ordering alone cannot guarantee an all-or-nothing append. After D and M but before I, X is physically present in B, yet it is not part of the user-visible file because the inode cannot reach B. A checker such as `fsck` can traverse inodes, compare their referenced blocks with the allocation bitmap, and reclaim the leaked B. It cannot infer that the user intended to append X or reconstruct a committed append from this state.

Question 2 - Redo-journal recovery

QUESTION

Use the same append, but now the file system has a **metadata-only redo journal**. Its protocol is:

```
A. Persist data block D.
B. Persist after-images M' and I' in the journal.
C. Persist a valid COMMIT record.
D. Copy M' and I' to their home locations.
E. Reuse the log space only after D is complete.
```

Each letter completes durably before the next begins; the two home copies in step D may happen in either order. Before C, neither home metadata block is changed. Recovery replays every transaction with a valid COMMIT.

1. A crash occurs after B but before C. What does recovery do, and what is the logical state of the file?
2. A crash occurs after C but before either home copy. What does recovery do?
3. A crash occurs after exactly one of the two home copies. What does recovery do, and why must redo be idempotent?
4. Why does this metadata-only protocol insist on A before C? What could go wrong if the data block were still unpersisted when COMMIT became durable?

ANSWER

(1) A crash occurs after B but before C: No valid COMMIT record exists, so recovery ignores this uncommitted journal transaction. It need not physically erase the journal records or undo the already-persisted data write in physical block B. The home inode and bitmap remain in their old states: B is logically free, and the append is absent from the file. X may remain physically in B until a future allocation initializes or overwrites it.

(2) A crash occurs after C but before either home copy: a valid COMMIT record exists -> execute the D, E operations again (copy M' and I' to their home locations and reuse the log space after the completion of copying); the append will be complete after the recovery.

(3) A crash occurs after exactly one of the two home copies a valid COMMIT record exists -> execute the D, E operations again (copy M' and I' to their home locations and reuse the log space after the completion of copying); the append will be complete after the recovery. the redo must be idempotent because after recovery, the disk does not know which step of the append operation is reached. it just know that it has reached the COMMIT milestone. for the sake of safety, it must assume that anything currently at their home locations is unreliable. so the recovery procedure has to be idempotent.

(4) if the protocol is: C -> A -> B -> D -> E if a crash happened after C, in the recovery stage, the disk cannot infer from the journal which stage has been reached; if the crash happens before D, the old copy is intact at home locations while after-images may not be complete; if the crash happend after B, the new copy is complete in after-images, but the data at home locations may be partially written. In a word, the version stored both in after-images and at home locations are not guaranteed to be valid/complete.

This C-first counterexample shows why the journal after-images (phase B) must be complete and durable before the COMMIT record: an early crash can leave a commit marker with incomplete log contents. A later crash during phase D is different. Once phase B has completed, the committed after-images are available to redo any partially updated home metadata; partial checkpointing is exactly what redo journaling tolerates.

To isolate the separate requirement that file data (phase A) precede COMMIT, keep the metadata log correctly ordered but try `B -> C -> A -> D -> E`. A crash after C but before A leaves a complete metadata journal and a durable COMMIT, while physical data block B still contains old bytes rather than X. Recovery replays M' and I', making the inode point to block B, but a metadata-only journal has no copy of X to restore. The file could therefore expose stale data. Ordered-data journaling prevents this by making phase A durable before phase C.

Question 3 - Replacing a complete file

QUESTION

`/d/report` initially names an old, complete, durable file. An application creates `/d/.tmp` in the **same directory and file system**, writes a complete new report, and uses:

```
fsync(temp_fd)
rename("/d/.tmp", "/d/report")
fsync(directory_fd_for_/d)
```

Assume all calls succeed, the device honors flushes, and this toy file system recovers a rename as **either the entire old binding or the entire new binding** if a crash occurs before the directory `fsync()`. Do not assume the new binding is durable until that `fsync()` succeeds.

1. Why is writing directly over /d/report insufficient if readers must see a complete old or complete new version?
2. What can /d/report name after recovery if the crash occurs (a) after fsync(temp_fd) but before rename, (b) after rename but before fsync(directory_fd_for_/d), and (c) after the directory fsync succeeds?
3. Explain the distinct roles of fsync(temp_fd), rename, and fsync(directory_fd_for_/d). What does an already-open fd for the old report continue to reference after the rename?

Review clarification: `fsync(temp_fd)` makes the temporary file's contents durable, but does not by itself guarantee that the directory entry `/d/.tmp` survives a crash before the rename. The old `/d/report` binding remains the relevant durable name at that point.

ANSWER

(1) This is a file-content atomicity problem, not necessarily corruption of the file system's structural metadata. Writing directly over `/d/report` can expose a mixture of old and new bytes to concurrent readers even without a crash. With write-back caching, a crash may also leave a mixed old/new version on storage. Writing a complete temporary file and then renaming it switches the pathname between complete versions instead of updating the visible file in place.

(2) (a) after fsync(temp_fd) but before rename: After recovery, `/d/report` still names the complete old version. The survival of `/d/.tmp` is not guaranteed because `fsync(temp_fd)` does not necessarily persist its directory entry. If that name was not persisted, the temporary file's flushed inode and data may be unreachable after recovery and subsequently reclaimed; if the name did persist, `/d/.tmp` names the complete new version. (b) After `rename`, before `fsync(directory_fd_for_/d)`: The problem states: crash recovery treats rename as either fully old binding OR fully new binding. So after recovery, `/d/report` either names the old file OR names the new report (the former `.tmp`). The rename may already have become durable through background activity, but without `fsync(directory_fd_for_/d)` the application has no guarantee that the new name binding survives a crash.

(c) After directory `fsync` succeeds: `/d/report` names the new complete report file. The directory entry change is persisted to disk, so the rename is fully durable.

(3) fsync(temp_fd) : Flush the new report's data and its inode to persistent storage. Guarantees the new file is fully written and durable on disk, before we touch the directory. We make sure the new content is complete before making it visible.

rename: Atomically replace the directory entry for `report` in the live namespace. Once `rename()` returns successfully, new opens of `/d/report` reach the new file; a concurrent lookup cannot observe a missing or partly replaced name. The old-or-new choice applies to crash recovery before the directory `fsync()`, not to the successful live rename. An fd opened for the old report before the rename continues to refer to the old inode.

fsync(directory_fd_for_/d): Flush the updated directory block(s) holding the renamed entry to disk. This makes the name binding change durable.

Lesson 21 - Networking foundations

Question 1 - Connection and scheduler state

QUESTION

A single-core machine has server thread S, client thread C, and unrelated worker W. S has already created a listening socket. Initially S is running, while C and W are ready. No completed connection is queued.

1. S calls blocking accept(). Describe the user/kernel transition, S's resulting thread state, and what the scheduler must do.
2. The scheduler runs C. C calls connect(), and eventually the connection is established and queued for S. What state can S enter? Must it immediately run?
3. When S's accept() returns, how many relevant socket descriptors does S now have? Which one would S use to read C's request? What happens to existing connections if S later closes only its listening socket?
4. C calls write(fd, request, n) and it returns n. State two things this guarantees and two things it does not guarantee.

ANSWER

(1) S makes a syscall `accept()`, so it changes from user mode to kernel mode As there is no pending connections, thread S will block the scheduler must pick a ready thread (C or W) to run next on the CPU; that thread can then return to user mode

(2) thread S will be ready after the connection is established and queued it does NOT necessarily run

(3) S now has two relevant socket descriptors: the listening socket and the newly accepted connected socket. S uses the latter to read C's requests. Closing only the listening socket prevents S from accepting new connections through that socket; it does not close already accepted connections.

(4) guarantees: the call reports success for exactly n bytes those n bytes have been accepted into the local socket's outgoing byte stream

doesn't guarantee: the bytes have already been transmitted to or received by the server the server application has read, processed, or durably stored the request

---

Question 2 - Framing a TCP stream

QUESTION

A protocol uses a **two-byte unsigned big-endian length**, followed by that many payload bytes. The maximum valid payload length is 1024. A peer sends two frames whose combined byte stream is:

```
00 03 41 42 43  00 02 78 79
```

The receiver calls `read()` with an eight-byte buffer repeatedly. In one possible execution, the calls return these byte groups:

```
read 1: 00 03 41
read 2: 42 43 00 02 78
read 3: 79
```

1. Decode the two payloads. After each read, say which complete frames can already be delivered to the application and which bytes must be retained.
2. Sketch read_exact(fd, buffer, n) in pseudocode. Explain how it handles a short positive read, interruption (EINTR), and EOF before all n bytes arrive.
3. Now suppose the connection instead reaches EOF after the first eight stream bytes. Is the second frame valid, absent, or truncated? Explain.
4. Would replacing the two-byte length with a single write() per message remove the need for framing? Why?

ANSWER

(1)

```
read 1: 00 03 41
decode: 00 03   -> the 1st payload has 3 bytes
        41      -> payload 1: 41 (incomplete, should be 3-byte-long)
                   the byte 41 must be retained as the payload 1 is incomplete

read 2: 42 43 00 02 78
decode: 42 43   -> payload 1: 41 42 43 (complete, is already 3-byte-long)
                   can deliver payload 1 to the application already
        00 02   -> the 2nd payload has 2 bytes
        78      -> payload 2: 78 (incomplete, should be 2-byte-long)
                   the byte 78 must be retained as the payload 2 is incomplete

read 3: 79
decode: 79      -> payload 2: 78 79 (complete, is already 2-byte-long)
                   can deliver payload 2 to the application already
```

The payloads are `ABC` and `xy` in ASCII. Once a length prefix has been parsed, the receiver must retain that framing state even if it no longer retains the prefix bytes themselves.

(2)

```
ssize_t read_exact(int fd, unsigned char *buffer, size_t n) {
    /*
    return n if success
    return a non-negative value less than n if encountered EOF before n bytes
    return -1 if failed
    */
    size_t offset = 0;

    while (offset < n) {
        ssize_t bytes_read = read(fd, buffer + offset, n - offset);
        // handles a short positive read by calling 'read()' for multiple times
        // until it reads n bytes in total

        if (bytes_read < 0) {
            if (errno == EINTR) {
                // read failure due to an interruption
                // retry read and continue
                continue;
            } else {
                // read failure due to other reasons
                return -1;
            }
        } else if (bytes_read == 0) {
            // encountered EOF, there are no more bytes to be read
            // cannot reach the target of n bytes
            return (ssize_t)offset;
        } else {
            // read normally, but does not reach n bytes
            offset += bytes_read;
            continue;
        }
    }

    return (ssize_t)n;
}
```

The caller must interpret an early EOF using context: EOF before any bytes of a new header can mean a clean end of the stream; EOF during a header or body means a truncated frame.

(3) The second frame is truncated, not absent: its prefix says that the payload has two bytes, but only one payload byte arrived before EOF. The prefix itself is well formed; the complete frame is not available.

(4) no. TCP is a byte sequence. if using `write()` without framing, the receiver can not determine which part of the data belongs to the first payload and which belongs to the other (does not even know the number of payloads!). the read pattern is arbitary and there's no message of boundary.

Question 3 - A stalled server

QUESTION

A server handles clients in one thread. It accepts A, then calls a helper that waits for A's **four-byte request header**. A sends just one byte and pauses indefinitely. Meanwhile B connects and sends a complete request.

1. Where can the server thread block? Can B's TCP connection be established even though the application has not accepted or serviced it yet? State any queue-capacity assumption you need.
2. Why is this not a failure of TCP's reliability guarantee?
3. Give two different server designs that could let B make progress despite A's stall. For each, name one cost or complication.
4. If this application changed from TCP to UDP, would the same "one byte of a four-byte datagram header now, three bytes later" situation occur in the same way? Explain the message-boundary difference, without claiming UDP solves all server stalls.

ANSWER

(1) The helper waits for exactly four header bytes. An ordinary first `read(fd, buf, 4)` may return the one byte A sent; a subsequent `read()` then blocks waiting for the other three. Since A pauses indefinitely without closing its sending direction, the helper sees neither a complete header nor EOF.

B's TCP connection can be established as long as the completed-connection queue has capacity. The handshake is handled by the kernel, so it can complete even while the server thread is blocked on A; accepting and servicing B must wait for the application.

(2) TCP provides an ordered, reliable byte stream while the connection works; it cannot make A send the remaining bytes or guarantee timely application progress. A's pause is an application-level stall, not a violation of TCP reliability. If the connection later fails, the application message may remain incomplete.

(3) first design: Use `fork()` after each successful `accept()`: the parent closes its copy of the accepted socket and continues its accept loop; the child closes its copy of the listening socket and handles that connection. A's stall does not prevent the parent from accepting and serving B through another child. The cost is process-creation, memory, and scheduling overhead.

second design: Enforce a finite read timeout and close A if it expires. This lets the single-threaded server serve B eventually, but only after the timeout; it does not provide concurrent progress. The timeout must actually interrupt the blocking wait, and it may close a legitimate slow connection.

(4) TCP carries a byte stream, whereas UDP preserves datagram boundaries. If the sender puts the four-byte header in one UDP datagram and the receive buffer is large enough, the receiver gets that datagram as a whole or does not receive it; it will not receive one byte now and the remaining three bytes later from the same datagram. A separately sent one-byte datagram is instead a complete but malformed request. UDP can still lose datagrams or leave a single-threaded server waiting for one.

Lesson 22 - RPC and distributed failure

Question 1 - One timeout, multiple histories

QUESTION

A durable server stores a counter initially equal to `10`. Client C sends `INCREMENT` with request ID `(C, 7)`. C receives no reply before its timeout. Assume there are no other clients.

1. Give two distinct event histories consistent with C's timeout: one in which the counter remains 10, and one in which it becomes 11.
2. C now blindly retries the request, and the retry executes successfully and returns a reply. What counter values are possible afterward? State the assumptions behind each value.
3. Suppose the original write() on C's TCP socket returned the full request length. Does that eliminate either history from part 1? Explain.

4. A programmer proposes sending a C pointer to a struct as the RPC argument. What must the client stub send instead, and why?

ANSWER

(1) history 1: the server crashes before it executes the increment operation / does not even receive the request in this case, the counter remains 10

history 2: the server durably executes the increment, but the reply is lost or delayed (the server may also crash after committing the effect) in this case, the counter becomes 11 even though C times out

(2) possibility 1: counter = 11 after the reply the original request never executes, and only the retry executes

possibility 2: counter = 12 eventually (possibly already when the retry replies) the original request executes, but its reply is lost or delayed; the blind retry also executes. If the original request remains in flight when the retry replies, the counter can be 11 at that instant and become 12 when the original executes later.

(3) No. A full-length return from `write()` means the local kernel accepted those request bytes into the socket's outgoing stream. It does not prove that the server application received the request or executed the operation, so neither timeout history is eliminated.

(4) The client stub must serialize the struct's values and any pointed-to data into an agreed byte representation, including necessary type and length information. Sending the numerical pointer value is useless because that address is meaningful only in the client's address space.

Question 2 - Duplicate detection across threads and crashes

QUESTION

A multithreaded RPC server implements `INCREMENT`. The counter is durable. Requests have unique `(client ID, sequence number)` identifiers. Two server threads receive copies of the **same** request `(C, 41)` close together because C retried.

1. Design the minimum logical states the server needs for each request ID. Explain how the two threads avoid both executing the increment.
2. The first execution changes the counter from 20 to 21, but its reply is lost. What should the server return when (C, 41) is received again? Why should it not recompute a response from the counter's current value?
3. Now assume the counter is durable but the duplicate table is volatile. The server crashes after the first execution and restarts before the retry. What can go wrong?
4. State the durability/atomicity invariant needed to preserve an at-most-once effect across that crash. You need not design a full database or consensus protocol.

ANSWER

(1) three logical states at minimal are needed for each request ID: UNSEEN, IN_PROGRESS, DONE

when a new request with request identifier = idx arrives: case 1: idx.state = UNSEEN the request arrives for the first time, we ought to execute it check and change idx.state from UNSEEN to IN_PROGRESS atomically

case 2: idx.state = IN_PROGRESS the same logical request is already executing, possibly in another thread. Do not execute it again. The duplicate can wait for the original result or receive a defined "retry later" response; silently ignoring it without a response policy could prevent client progress.

case 3: idx.state = DONE the request has completed before. Do not execute it again; resend the saved original reply.

as the operation of state is atomic, only one thread will treat the request as UNSEEN and execute increment operation.

(2) The server should resend the saved original reply, which reports the new counter value `21`, without incrementing again. The retry has the same identifier `(C, 41)` and is already in `DONE`. Recomputing a reply from the current counter could give a different result if later operations changed it.

(3) If the server crashes after the first execution and restarts before the retry, the durable counter retains the increment but the volatile duplicate table loses the completion record. The server then treats `(C, 41)` as new and may increment a second time.

(4) invariant: The counter effect and the durable `DONE` record, including the saved original reply, must commit atomically. After recovery, either neither exists or both exist. Persisting the duplicate table separately from the effect still leaves a crash window.

Question 3 - Is a retry actually safe?

QUESTION

Assume a key/value server supports these operations:

```
SET(k, v)       → returns OK
INCREMENT(k)    → returns the new value
DELETE(k)       → returns whether k existed
GET(k)          → returns the current value
```

1. With no other clients, classify each operation: can repeating it change the final state? Can it change the returned result? Explain any assumptions.
2. Initially k = 0. A's SET(k, 1) executes but its reply is lost; then B successfully performs SET(k, 2); finally A blindly retries its original SET(k, 1). What is the final value? Why doesn't state-idempotence protect B here?
3. How would a correctly implemented request-ID scheme change the outcome in part 2? What identifier should A use for the retry, and what identifier should it use for a later, genuinely new SET(k, 1)?

ANSWER

(1) assumption: repeat only the exact operation, no interleaving

- SET(k, v): repeating it does not change the final state or the returned OK, assuming both calls succeed.
- INCREMENT(k): repeating it changes both the final state and the returned new value.
- DELETE(k): repeating it does not change the final state, but if k initially existed, the replies differ (true, then false). If it was initially absent, both replies are false.
- GET(k): repeating it changes no state and returns the same value under the no-interleaving assumption.

(2) the final value = 1 State-idempotence only guarantees that consecutive repetitions have no additional state effect. Here B's `SET(k, 2)` occurs between A's first execution and A's blind retry, so the retry overwrites B.

(3) Use an identifier such as `(client ID, sequence number)`. A's retry must reuse the original identifier, so the server returns its saved `OK` without executing `SET(k, 1)` again. B's later write remains in place: the final value is `2`. A must use a new identifier (a different sequence number) for a later, genuinely new `SET(k, 1)`.

Lesson 23 - Protection and security

Question 1 - ACLs, handles, and revocation

QUESTION

File F's ACL allows Alice to read it but gives Bob no rights. Alice opens F read-only and passes the resulting open descriptor to Bob through a Unix-domain socket. Assume the OS permits this transfer and follows ordinary Unix open-descriptor semantics: later ACL changes do not retroactively revoke an already-open descriptor.

1. Can Bob open F by pathname using his own credentials? Can he read through the descriptor Alice passed? Explain why these answers differ.
2. Alice closes her descriptor. Later, an administrator removes Alice's read permission from F's ACL. What happens to Bob's existing descriptor? What happens if Alice now tries to open F again?
3. Why would guessing the integer value of Alice's descriptor, without the kernel-mediated transfer, not give Bob the same authority?
4. Suppose the administrator needs revocation to stop Bob's future reads promptly. Suggest a design other than handing Bob a permanently usable bare descriptor, and state one cost of that design.

ANSWER

(1) Bob cannot open F by pathname using his own credentials because F's ACL gives him no rights. However, he can read through the already-open, read-only descriptor Alice deliberately passed to him. Alice was authorized when she opened F, and the kernel-mediated transfer gives Bob a reference to that open object. This is delegation of authority, not a confused-deputy case; Bob does not perform a new pathname-based `open()`.

(2) After Alice closes her descriptor, Bob's own descriptor remains valid and he can still read F. ACL changes do not retroactively revoke already-open descriptors under the stated ordinary Unix semantics. Bob's reference keeps the open file description alive. After Alice's read permission is removed, she cannot newly open F by pathname.

(3) The integer value of a file descriptor is not globally meaningful; it is interpreted through the calling process's descriptor table. The same integer can refer to different open file descriptions in different processes. Bob needs an actual kernel-mediated transfer of a reference, not a guess of Alice's descriptor number.

(4) One design is a revocable per-recipient grant: the kernel tracks Bob's handle and checks its grant state on each read. An administrator can invalidate Bob's grant without closing the underlying file for everyone. This requires grant metadata, synchronization, and per-access checking. Rechecking only F's ACL against Bob would not work for this scenario: Bob has no ACL permission even before revocation, so it would defeat the intended delegation.

Alternatively, Alice or a trusted service could retain the descriptor and mediate Bob's reads, checking a revocable grant on every request. This adds IPC overhead and makes Bob depend on the mediator's availability.

Question 2 - Two check/use races

QUESTION

A privileged service accepts a pathname in a directory whose entries the caller can change. It calls `stat(path)`, checks that the reported owner is the caller, and later calls `open(path, O_RDONLY)` and returns the file's contents. The caller cannot directly read `/private/key`, but the service can.

1. Give a concrete interleaving in which the ownership check succeeds yet the service opens /private/key. State what the attacker changes and when.
2. Would a mutex around stat and open within the service necessarily prevent this attack? Identify who is outside that mutex.
3. Describe the security property a safer design must achieve: what object must be authorized, whose authority must be checked, and how must that object remain linked to the later read? You need not give platform-specific code.
4. Separately, a kernel syscall checks the contents of a user buffer and later copies from the same pointer. Another user thread can modify the buffer in between. Why is a user-provided mutex insufficient, and what property should the kernel's copying/validation strategy provide?

ANSWER

(1) attacker:

- (1) create a safefile owned by himself
- (2) submit the path to the service; the service calls stat(safefile), which passes its ownership check
- (3) after the service's stat() returns but before its open(), unlink safefile and replace that directory entry with a symlink to /private/key
- (4) the privileged service calls open("safefile", O_RDONLY), follows the new symlink, and actually opens /private/key

(2) Mutex inside service won't help. You can't expect a malicious program to obey the rules and `acquire(&myLock)` to ensure safety. A mutex can only serialize code inside the service process. The mutex only stops multiple threads within the service from interleaving; it cannot block the attacker from modifying filesystem objects between service's stat and open.

(3) We need to authorize the actual object opened, under the original caller's authority rather than merely the service's stronger authority. Authorization and the later read must refer to the same opened object through a stable handle, not two separate resolutions of a mutable pathname. The handle stabilizes object identity; it does not by itself freeze the file's contents.

(4) A user-provided mutex is insufficient because the kernel cannot trust a malicious user thread to acquire it. The kernel should copy the data into kernel-owned memory, then validate and use that same copy, so later user writes cannot

change the bytes being used. Pinning user pages may address residency or unmapping, but it does not prevent another user thread from modifying their contents; pinning alone does not solve this TOCTOU race.

Question 3 - The confused deputy

QUESTION

A backup service runs with permission to read `/private/key`; Bob has no such permission. The service accepts `copy(source, destination)` requests. It verifies that the requester is an authenticated user, then opens `source` using **the service's** credentials and copies its contents to `destination`. Bob asks it to copy `/private/key` into a file Bob can read.

1. Identify the deputy, the authority it misuses, and the missing authorization decision. Is this attack necessarily a TOCTOU race?
2. Give two defenses based on different principles-for example, checking the caller's authority, delegating a restricted capability, or reducing the service's privileges. Explain the security invariant each defense aims to establish.
3. Suppose the service opens /private/key, then drops its privileged identity, but keeps the already-open descriptor. Under the ordinary descriptor semantics from Q1, is dropping identity after opening sufficient to prevent this copy? Explain.

ANSWER

(1) Deputy: the backup service. It acts on behalf of Bob but holds extra privileges of its own. Authority misused: the service uses its own read permission over `/private/key` to fulfill Bob's request. Missing authorization: the service authenticates Bob but does not check whether Bob is authorized to read the source and write the destination. This attack does not require a TOCTOU race. TOCTOU means that a checked condition can change before use; here the service consistently uses its stronger authority to perform Bob's unauthorized request.

(2) defense 1: checking the caller's authority For this user-directed `copy` API, authorize the caller's read access to the actual source and write access to the actual destination. If Bob lacks either right, reject the request even if the service has the right itself. Invariant: This API copies only between objects that the requester is authorized to read and write, using stable object references so the checks apply to the objects actually used. A different privileged backup policy would need its own explicit authorization rule; authentication alone is not enough.

defense 2: reducing the service's privileges Handle Bob's request in a worker running with Bob's privileges *before* opening the source, and do not pass it any already-open privileged descriptors. If the service needs elevated rights for other duties, separate those duties from this worker. Invariant: The worker has no authority, whether through its identity or inherited handles, to read `/private/key`; it cannot misuse privilege it does not possess.

(3) Dropping identity after opening is not sufficient. Under the stated ordinary descriptor semantics, the service opened the file while privileged and retains a readable descriptor. It can still read through that descriptor and copy the contents to Bob's file after dropping its privileged identity.

Lesson 24 - Integrated review

Question 1 - Pipe EOF, scheduling, and a parent that waits too early

QUESTION

On a single-core machine, parent P creates a pipe and forks writer W and reader R. R closes its inherited write end; W closes its inherited read end. P closes its read end but **keeps its write end open**. Initially the pipe is empty.

R runs first and calls `read(pipe_read, buf, 4)`. Later W runs, writes the two bytes `"OK"`, closes its write end, and exits. The scheduler eventually runs R. R calls `read(pipe_read, buf, 4)` a second time after its first read returns.

1. Trace R's state from its first read() until W writes. When W writes, must R run immediately? Distinguish the relevant mode transitions from context switches.
2. How many bytes may R's first read() return in the described execution? Why need it not wait for all four?
3. Why can R's second read() block even though W has exited? What event would allow it to return 0?

4. Suppose P calls waitpid(R, ...) before closing its write end and performs no other actions until waitpid returns. Explain the resulting wait dependency and give the minimal ordering fix.

ANSWER

Parts (1)-(4).

Analysis

Initially, process P creates a pipe:

```
fd1_p(write) -> open file description 1 -> the pipe
fd2_p(read)  -> open file description 2 -> the pipe
```

Then, P forks writer W and reader R:

```
fd1_p(write) -> open file description 1 -> the pipe
fd2_p(read)  -> open file description 2 -> the pipe
fd1_w(write) -> open file description 1 -> the pipe
fd2_w(read)  -> open file description 2 -> the pipe
fd1_r(write) -> open file description 1 -> the pipe
fd2_r(read)  -> open file description 2 -> the pipe
```

Afterwards, R closes its inherited write end; W closes its inherited read end; P closes its read end but keeps its write end open:

```
fd1_p(write) -> open file description 1 -> the pipe
fd1_w(write) -> open file description 1 -> the pipe
fd2_r(read)  -> open file description 2 -> the pipe
```

Next, R runs first and calls `read(pipe_read, buf, 4)`: the system call traps from user mode into kernel mode. Because the pipe is empty but at least one write end remains open, the kernel blocks R (**running -> blocked**) and schedules another runnable process (**a context switch**). Entering the kernel for a system call is a mode switch; it is not itself necessarily a context switch.

Later, W runs, writes the two bytes "OK", closes its write end, and exits: the write enters the kernel, adds data to the pipe, and wakes R (**blocked -> ready**). Waking R does not itself require an immediate context switch: W may continue running until it blocks, yields, exits, or is preempted. When the scheduler eventually runs R (**ready -> running**, a context switch), R's pending `read()` returns 2 with "OK"; it need not wait to fill the four-byte buffer. R returns to user mode before executing its next user instruction.

Eventually, R calls `read(pipe_read, buf, 4)` again, entering kernel mode. Because the pipe is empty while P still holds a write end open, R blocks again (**running -> blocked**), and the scheduler switches to another process. W's exit alone is insufficient for EOF. EOF is not a separate signal: a pipe read returns 0 when the buffer is empty and no write ends remain open. If P closed its write end, R could be awakened and its pending read could return 0.

If P calls `waitpid(R, ...)` before closing its write end and performs no further actions until `waitpid` returns, a deadlock occurs. P enters the kernel and blocks waiting for R to exit. R cannot finish its read and exit until P closes the last write end; P cannot close it while blocked in `waitpid`. Both processes are blocked, and the scheduler cannot resolve this circular wait merely by choosing a different process.

Question 2 - COW meets paging and I/O

QUESTION

Parent P has one resident **anonymous** virtual page V containing byte `0x2A`. It maps to frame F and is initially writable. P forks child C. The implementation uses copy-on-write (COW), so both mappings initially refer to F with write protection.

C writes `0x99` to V. Later C faults on an unrelated virtual page; memory is full, and the kernel's Clock policy selects **C's page containing** `0x99` as the victim. Assume that page is dirty, eviction writes it to swap, and C blocks until this I/O finishes. Eventually C accesses V again. Finally P writes to its own V.

1. Describe the mappings and contents immediately after fork, and immediately after C's first write. What caused C's write fault, and was disk I/O required for that fault?
2. During eviction, what data must be preserved, and why? When the swap write finishes, does the completion interrupt necessarily make C run immediately?
3. What happens when C accesses V after its page has been evicted? What value must C observe? What value does P still have before its own write?

4. After C separated from F, is a new frame logically required for P's later write to V? Explain what a COW implementation may do if P is F's only remaining owner.

ANSWER

(1)

Analysis

Stage 1 - Parent P has one resident anonymous virtual page V containing byte 0x2A;

```
virtual page V -> frame F (RW): byte 0x2A
```

Stage 2 - Parent P forks child C. (copy-on-write)

```
P: virtual page V -> frame F (R-, COW-protected): byte 0x2A
C: virtual page V -> frame F (R-, COW-protected): byte 0x2A
```

Stage 3 - Child C writes 0x99 to V.

```
P: virtual page V -> frame F : byte 0x2A
C: virtual page V -> frame G (newly allocated, RW): byte 0x99
```

Stage 4 - C faults on an unrelated virtual page and V is evicted

```
P: virtual page V -> frame F : byte 0x2A
C: virtual page V -> non-existing in memory, be evicted to a swap slot S in storage. C gets blocked until
    this I/O finishes
```

Stage 5 - C accesses V again

```
P: virtual page V -> frame F : byte 0x2A
C: load the page contents from swap slot S into an available frame H; virtual page V -> frame H (RW):
    byte 0x99. H might coincidentally be G, but that is not guaranteed.
```

Stage 6 - P writes to its own V

The COW protection causes C's write fault, but no disk I/O is required for that fault here: F is resident, so the kernel copies its contents into G in memory, then completes C's write. The later eviction of dirty G requires a swap write. (If allocating a frame for COW required evicting some other page, that separate memory-pressure step could involve I/O; it is not needed in this stated sequence.)

(2) During eviction, the dirty page contents, including byte 0x99, must be written to swap slot S before G is reused. The kernel also retains metadata (in the PTE and/or a supplemental page table) marking C's V nonresident and identifying S so it can be reloaded later; this metadata is not itself the page contents being written to swap. When the swap write finishes, the completion interrupt doesn't necessarily make C run immediately. It changes C's state from blocked to ready; the scheduler decides when to run it.

(3) When C access V after its page has been evicted, a page fault takes place as its page table entry marks it as invalid. During the page fault, the page will be loaded from the swap slot to the memory and the PTE will be valid and point to the newly allocated frame inside memroy. C must observe the new byte 0x99. P still has 0x2A before its own write because they now both have their own physical frame of virtual page V.

(4) No new frame is logically required. If P is F's only remaining owner, the kernel can make P's mapping writable without copying F. P's next write may still trigger a protection fault if the PTE remains read-only; its handler can simply change the permission and retry. Alternatively, the kernel may have restored write permission earlier. Any changed translation permissions require appropriate TLB maintenance.

Question 3 - A distributed file service with two different bugs

QUESTION

A privileged server offers `APPEND(path, bytes, request_id)`. Initially `/uploads/report` contains `"A"`. Client C is authorized to append to that file. Attacker M can change the directory entry named `/uploads/report`, but cannot directly write `/private/key`; the server can.

For authorization, the server first calls `stat(path)` and checks the object against C's rights, then later calls `open(path, O_APPEND)` using its own privileged credentials. Its RPC duplicate table has `IN_PROGRESS` and `DONE(saved_reply)` states. Assume a correct write-ahead-logged transaction can atomically commit **both** a file append and the corresponding durable `DONE` record.

C sends request `(C, 9)` to append `"B"` while the path still names the legitimate report. The server durably commits the transaction, including a saved reply “new length = 2,” but that reply is lost. The server crashes and recovers. C retries the **same** request ID. For duplicates already in `DONE`, the server returns the saved reply *before* doing another pathname lookup.

1. What are /uploads/report’s contents after recovery? What reply should C’s retry receive, and should the append run again?
2. If the duplicate table had instead been volatile while the append remained durable, and the pathname stayed unchanged, what could the retry do to the report?
3. For a new request ID (C, 10), M swaps /uploads/report to a symlink to /private/key between stat and open. Does the journal or duplicate table prevent the server from appending to the wrong object? Explain which security boundary fails.
4. State two separate invariants the service needs: one for crash-safe duplicate suppression, and one for authorization of the actual opened object. Do not propose a full implementation.

ANSWER

(1) After recovery, `/uploads/report`'s contents are "AB" because the change has been committed before the server crashed. C's retry should receive a reply with "new length = 2" and the request identifier. The append shouldn't run again because the server has performed the operation with the same request identifier before and that is recorded in the RPC duplicate table with `DONE(saved_reply)` state.

(2) If the duplicate table had instead been volatile while the append remained durable, the server will lost the duplicate table after the crash and it won't be able to know that it has performed the operation (C, 9) before. So when it received the retry request, it will find no record in its duplicate table and interpret the request as an unseen one. As a result, the server will perform the operation again, and the contents of `/upload/report` will be "ABB".

(3) Neither the journal nor the duplicate table prevents the server from appending to the wrong object. It is a typical TOCTOU race condition. The duplicate table won't prevent the error because it just checks whether the request has been carried out before to avoid duplication; it doesn't eliminate the race condition. The journal only makes writes atomic/durable, it does not fix the TOCTOU race.

the analysis of the problem: The server checks the authorization by using `stat()` on `/uploads/report` earlier than the `rename()/symlink()`. At `stat()`, the path pointed to a legal file, so the authorization passes; but when `open()` resolves the path again, the path now links to `/private/key`. The server opens and appends to the key using its own privileged credentials.

the failed boundary: the authorization check takes place at `stat()` while the usage happens later; authorization is tied to `path` instead of the actual `inode` of the file.

(4) two required invariants: **Crash-safe duplicate suppression invariant**: The file append and the request ID's `DONE(saved_reply)` record must become durable atomically: after recovery, either both are present or neither is. If they committed, re-submission of that request ID must return the saved reply without re-executing the append. This rules out a crash window in which the append is durable but its duplicate-suppression record is lost. **Authorization invariant**: The authorization check must validate the exact file object that is ultimately opened and modified, not just the object named by the path at an earlier separate stat lookup.